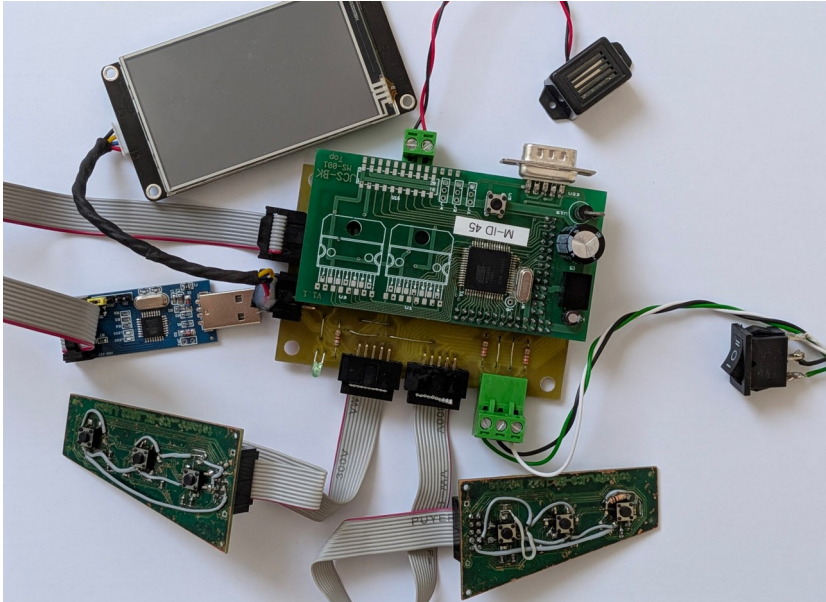


BERUFLICHES GYMNASIUM TECHNIK

JOHANN-CONRAD-SCHLAUN-BERUFSSKOLLEG



Dokumentation

Go-Kart Blinker Lenkrad Anzeige

Dean Schneider

Auftraggeber:	Michael Schmidt
Projektleitung:	Henry Otto
Projektnummer:	JCS-BK_1091_GO-KART
Jahrgang:	GYT26
Fach:	Projekt mit Elektrotechnik und Informatik
Verfasser:	Dean Schneider
Projektstart:	1. September 2025
Projektende:	20. März 2026
Letzte Änderung:	13. März 2026

Abstract

For several years, students at the technical high school have been working on upgrading a go-kart by integrating systems typically found in modern cars. The individual modules communicate via a CAN bus network, similar to that used in real vehicles. Two microcontrollers from the AVR family are used throughout the system. The task is divided into multiple projects – one for each student.

This document describes the development of the light modules and the cockpit module. There is also a video showcasing the development. The lighting system consists of four modules, one for each corner of the vehicle. The two front modules support upper beam, lower beam, and daytime running lights, while the two rear modules provide tail, brake, and reverse lights.

The cockpit module includes a display, a horn, and several buttons mounted on the steering wheel. It serves as the central interface between the driver and the vehicle systems. The display presents information such as speed, battery charge, current gear, light status, and other relevant data. Using the steering wheel buttons and switches, the driver can control things like the driving direction, light mode, blinker direction, horn and gear.

All required tasks of this project were successfully completed. The service provider (TIO) did not finish the Java-based frontend; however, the serial communication interface has already been implemented. Since the other projects have not yet integrated CAN bus communication into their modules, communication between the cockpit and the other systems is currently not possible. In addition, a 3D-printed case for the display and caps for the buttons could be developed in the future.

Kurzfassung

Seit mehreren Jahren arbeiten Schüler der gymnasialen Oberstufe Bereich Technik daran, ein Go-Kart durch die Integration von Systemen, die typischerweise in modernen Autos zu finden sind, aufzurüsten. Die einzelnen Module kommunizieren über CAN-Bus, ähnlich wie es in echten Fahrzeugen verwendet wird. Im gesamten System kommen zwei Mikrocontroller aus der AVR-Familie zum Einsatz. Die Aufgabe ist in mehrere Projekte unterteilt – eines für jeden Schüler.

Dieses Dokument beschreibt die Entwicklung der Lichtanlage, des Lenkrads und der Tachoanzeige. Des weiteren gibt es ein Video in dem die Entwicklung gezeigt wird. Die Lichtanlage besteht aus vier Modulen, eines für jede Ecke des Fahrzeugs. Die beiden vorderen Module unterstützen Fernlicht, Abblendlicht und Tagfahrlicht, während die beiden hinteren Module Rücklicht, Bremslicht und Rückfahrlicht liefern.

Das Cockpitmodul umfasst ein Display, eine Hupe und mehrere am Lenkrad angebrachte Taster. Es dient als zentrale Schnittstelle zwischen dem Fahrer und den Fahrzeugsystemen. Das Display zeigt Informationen wie Geschwindigkeit, Batterieladestand, aktueller Gang, Lichtstatus und andere relevante Daten an. Mit den Tasten und Schaltern am Lenkrad kann der Fahrer beispielsweise die Fahrtrichtung, den Lichtmodus, die Blinkerrichtung, die Hupe und den Gang steuern.

Alle gegebenen Aufgaben dieses Projekts wurden erfolgreich abgeschlossen. Der Dienstleister (TIO) hat das Java-basierte Frontend nicht fertiggestellt, jedoch wurde die serielle Schnittstelle bereits implementiert. Da die anderen Projekte die CAN-Bus-Kommunikation noch nicht in ihre Module integriert haben, ist eine Kommunikation zwischen dem Cockpit und den anderen Systemen derzeit nicht möglich. Darüber hinaus könnte in Zukunft ein 3D-gedrucktes Gehäuse für das Cockpit und Kappen für die Lenkrad-Taster entwickelt werden.

Inhaltsverzeichnis

Abstract.....	1
Kurzfassung.....	1
1 Einleitung.....	4
1.1 Projektziel.....	4
1.2 Hinweise zu diesem Dokument.....	4
1.3 Code Konventionen.....	4
2 Ergebnis.....	5
2.1 Team.....	5
2.2 CAN-Protokoll.....	7
2.3 Cockpit.....	9
2.4 Blinker-Module.....	13
2.5 Dateistruktur.....	14
2.6 CAN-Bus.....	16
2.7 Mikrocontroller.....	17
2.7.1 ATmega16M1.....	17
2.7.2 AT90CAN32.....	18
3 Entwicklung.....	19
3.1 Programmierer.....	19
3.2 Erste Schritte mit Masterplatine.....	20
3.3 Entwicklung mit HAL.....	21
3.4 Bluetooth-Modul.....	22
3.4.1 Test und konfigurieren mit USB-zu-Seriell-Konverter.....	22
3.4.2 Schaltung mit Mikrocontroller.....	24
3.5 Blinker-Modul.....	26
3.5.1 Berechnung des Timers.....	26
3.6 CAN-Bus.....	30
3.6.1 Anschluss.....	31
3.6.2 Beispiel-Programm.....	32
3.7 Erstes Display (Tacho).....	34
3.8 Cockpit-Display (Tacho).....	35
3.8.1 Befehle.....	35
3.8.2 Beispiel-Programm.....	37
3.8.3 Bibliothek Interface.....	38
3.9 Huppe.....	39
3.9.1 Erster Transistor.....	40
3.9.2 Finaler Transistor (IRL3803).....	41
3.10 Power-LED Dimensionierung.....	42
3.11 Lenkrad.....	43
3.12 Cockpit-Platine.....	44
3.12.1 Prüfprotokoll.....	48
3.13 Cockpit-Programm.....	50
4 Grundlagen.....	54
4.1 Kompilieren und Hochladen.....	54
4.2 UART Übertragung.....	55
4.2.1 Text übertragen.....	55
4.2.2 Rohdaten übertragen.....	55
4.2.3 Bluetooth Übertragung.....	55
4.3 CAN Sniffing (PCAN-USB).....	56
4.3.1 Empfangen.....	56
4.3.2 Senden.....	56

4.3.3 Fehlerbehebung.....	56
5 Fazit.....	57
5.1 Kosten.....	57
5.2 Reflexion.....	57
5.3 Ausbau.....	58
6 Eidesstattliche Erklärung.....	59
7 Anhang.....	60
7.1 Projekte der Vorgänger.....	60
7.2 Terminplan.....	61
7.3 Meilensteine.....	65
7.4 Tagesziele.....	66
7.5 Abbildungsverzeichnis.....	69
7.6 Schaltplanausschnitte.....	69
7.7 Tabellenverzeichnis.....	70
7.8 Quellenverzeichnis.....	70

1 Einleitung

Diese Dokumentation richtet sich an Personen, welche auf dem Wissensstand der 12. Klasse berufliches Gymnasium Technik sind. Die Themen sind nach ihrer Priorität für das Nachschlagen geordnet. Das bedeutet, dass zunächst der aktuelle Aufbau (das **Ergebnis**) beschrieben wird, dies ist geeignet für Techniker welche keine interesse in den internen Einzelheiten ist. Anschließend die **Entwicklung** hin zum aktuellen Aufbau, dies ist das Kapitel für Personen, welche das Projekt weiterführen oder warten möchten. Schließlich die **Grundlagen** zur Bedienung der Software.

1.1 Projektziel

Um überhaupt erst mit dem Projekt zu beginnen müssten alle erhaltenen Hardware Produkte auf Funktion getestet. Die Dokumentationen der Vorgänger waren dabei oft falsch oder nicht hilfreich.

- Programm für Blinker-Module entwickeln
 - animierter Blinker
 - über UART steuerbar
 - über CAN-Bus steuerbar
- Programm für Cockpit
 - Anzeige auf Display und Einlesen von Tastern
 - Kommunikation über CAN-Bus
- Platine für Cockpit welche das Display, die Taster, die Hupe und weitere Dinge verbindet.

Alle Ziele wurden erreicht und sind unter dem Kapitel 2 Ergebnis (siehe S. 5) zu finden.

1.2 Hinweise zu diesem Dokument

Das hier genannte Cockpit-Display wird in anderen Dokumentationen Tacho, Tachodisplay oder einfach nur Display genannt.

Jeder Schaltplan in dieser Dokumentation entspricht der realität, das heißt Änderungen an der realen Cockpitplatine sind in den Schaltplänen aktualisiert, um Verwechslungsgefahr zu vermeiden.

Gesamte Anleitung für Linux um genau zu sein für die Arch Distribution.

MCU (Microcontroller Unit) entspricht Mikrocontroller.

Hexadezimalzahlen werden durch ein „h“ gekennzeichnet (z. B. FFh oder 003Bh).

1.3 Code Konventionen

- Einheitliche Einrückung mit 4 Leerzeichen (keine Tabs)
- Maximale Zeilenlänge von 80 Zeichen
- Variablen und Funktionen: snake_case
- Konstanten: UPPER_CASE
- Aussagekräftige Namen statt Abkürzungen (calculate_total statt calc)
- Kommentare richten sich nach dem Google Style C++ Guide (<https://google.github.io/styleguide/cppguide.html#Comments>)

2 Ergebnis

Unter dem Kapitel Entwicklung befinden sich ausführliche Beschreibungen der im Folgenden genannten Themen.

2.1 Team

Dieses Projekt wird in enger Zusammenarbeit mit dem Team realisiert. So muss die Kommunikation abgesprochen werden. Die Verteilung der Aufgaben sieht wie folgt aus:

Name	Aufgabe
Henry Otto	Motoransteuerung und Auslesung der Pedalen
Iuri Merklein Muniz	Elektrische Bremse und Gangschaltung
Tammo Lohe	Ladeelektronik, Batterie und Solar
Dean Schneider	Lenkrad, Cockpitdisplay und die Beleuchtung

Tabelle 1: Team

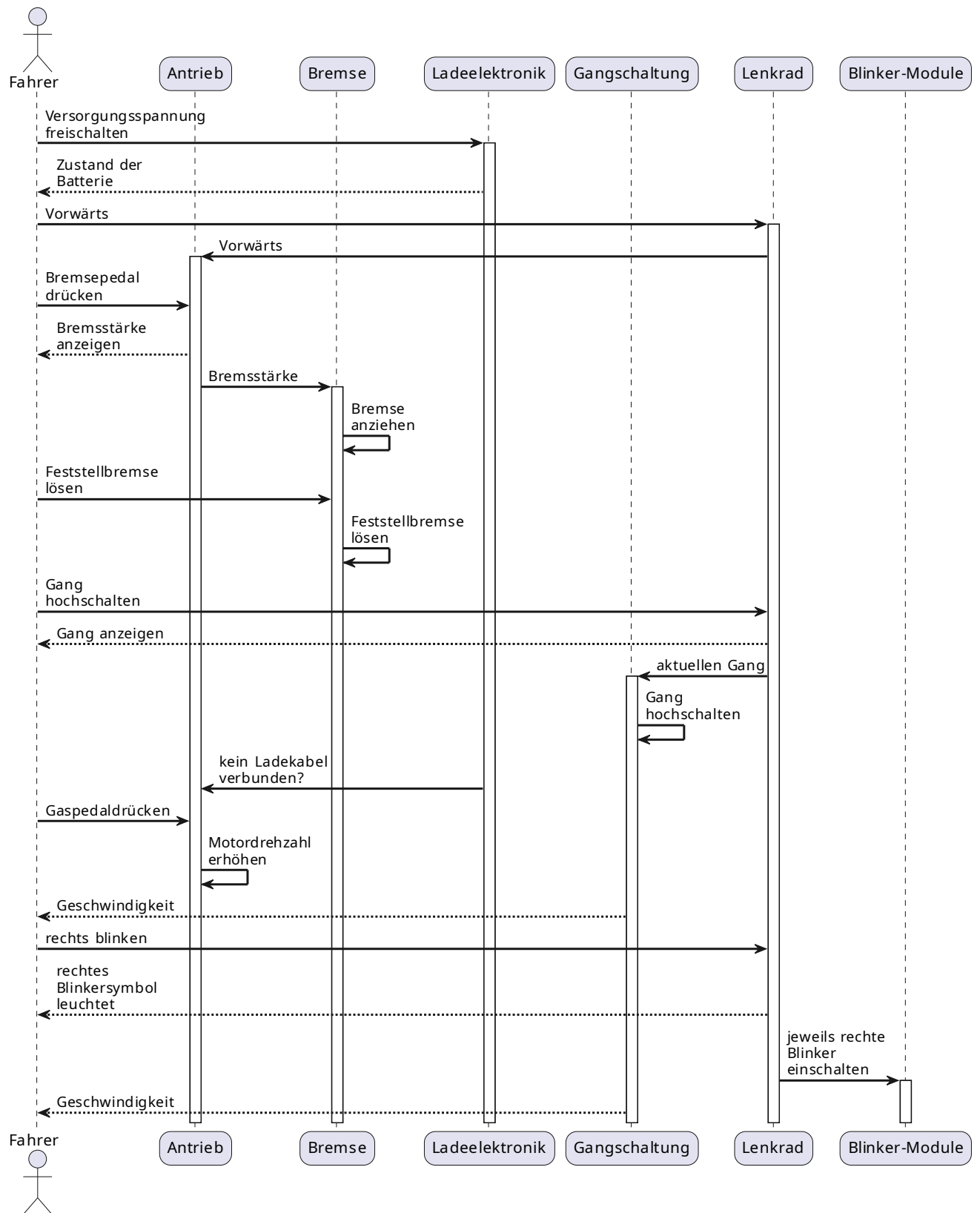


Abbildung 1: Kommunikation zwischen den Projekten im Sequenzdiagramm

Nachrichten an den Fahrer werden durch das Display visualisiert. Die meisten Nachrichten werden in einem Interval gesendet und sind in Abbildung 1 nur vereinfacht dargestellt, die gesamte Kommunikation ist dem CAN-Prokoll zu entnehmen (siehe 2.2 CAN-Protokoll, S. 7).

2.2 CAN-Protokoll

Erstellt von Dean Schneider (GYT26) Version 0.4.0

Legende:

MSB least significant bit

LSB most significant bit

Blinker senden ihre CAN-ID in den 2. (MSB) bis 5. Byte der Statusnachrichten

Rot markiert, dass nur ein Bit 1 sein darf, zusätzlich wird in Gruppen unterteilt, z. B. (A) oder (B).

an CAN-ID	Interval	Gruppe	Nachricht an Gerät	von Gerät	1. Datenbyte							
					Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0x703	~1 Hz	Status	Cockpit	jedem Blinker					voll Licht oben	leicht Licht oben	Blinker	Licht unten
0x702		Status	Antrieb	Bremse	MSB	Bremsenstärke von 0 bis 255, großer Wert für starke Bremsung						LSB
0x701		Status	Antrieb und Cockpit	Bremse / Gangschaltung	0: Stehen 1: Fahren	(A) Fußbremse	(A) Handbremse		(B) 3. Gang	(B) 2. Gang	(B) 1. Gang	(B) Neutralgang
0x605		Anzeige	Cockpit	Gangschaltung	MSB	Geschwindigkeit von 0 bis 255						LSB
0x604		Anzeige	Blinker vorne rechts	Cockpit					Fernlicht	Abblendlicht	Blinker	Tagfahrlicht
0x603		Anzeige	Blinker vorne links	Cockpit					Fernlicht	Abblendlicht	Blinker	Tagfahrlicht
0x602		Anzeige	Blinker hinten rechts	Cockpit					Bremslicht	Rücklicht	Blinker	Rückfahrlicht
0x601		Anzeige	Blinker hinten links	Cockpit					Bremslicht	Rücklicht	Blinker	Rückfahrlicht
0x302		Ladeelektronik	Ladeelektronik und Cockpit	Antrieb mit Pedale	MSB	Strom von 0 bis 255						LSB
0x301		Ladeelektronik	Antrieb und Cockpit	Ladeelektronik			Batterieladen	4/4 Batterie	3/4 Batterie	2/4 Batterie	1/4 Batterie	leere Batterie
0x202		Gangschaltung	Gangschaltung	Cockpit					3. Gang	2. Gang	1. Gang	Neutralgang
0x201		Bremse	Bremse und Cockpit	Pedale (Antrieb)	MSB	Bremsenstärke von 0 bis 255, großer Wert für starke Bremsung						LSB
0x101		Antrieb	Antrieb	Cockpit							0: Vorwärts 1: Rückwärts	0: Stop 1: Start
0x0..		Fehler										

Tabelle 2: CAN-Protokoll

Dies zeigt alle Nachrichten innerhalb des Controller-Area-Network. Damit man weiß von welchen Blinker-Modul die Statusmeldung kam, sendet der jeweilige Blinker die eigene CAN-ID.

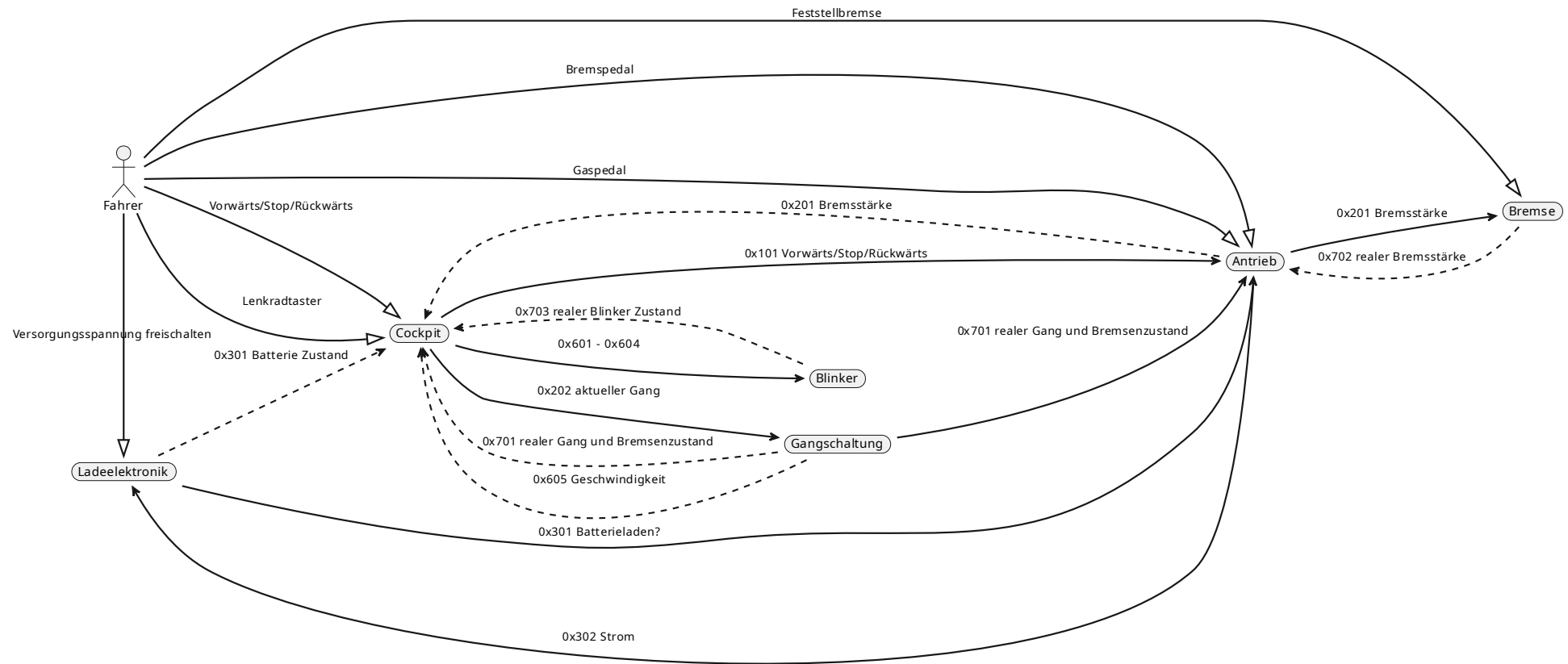


Abbildung 2: Kommunikationsdiagramm des CAN-Protokolls

Dieses Diagramm zeigt alle CAN-Nachrichten aus Tabelle 2 (siehe S. 7) und an wenn sie gerichtet sind. Gestrichelte Pfeile sind dabei Status- oder Bestätigungsnachrichten.

2.3 Cockpit

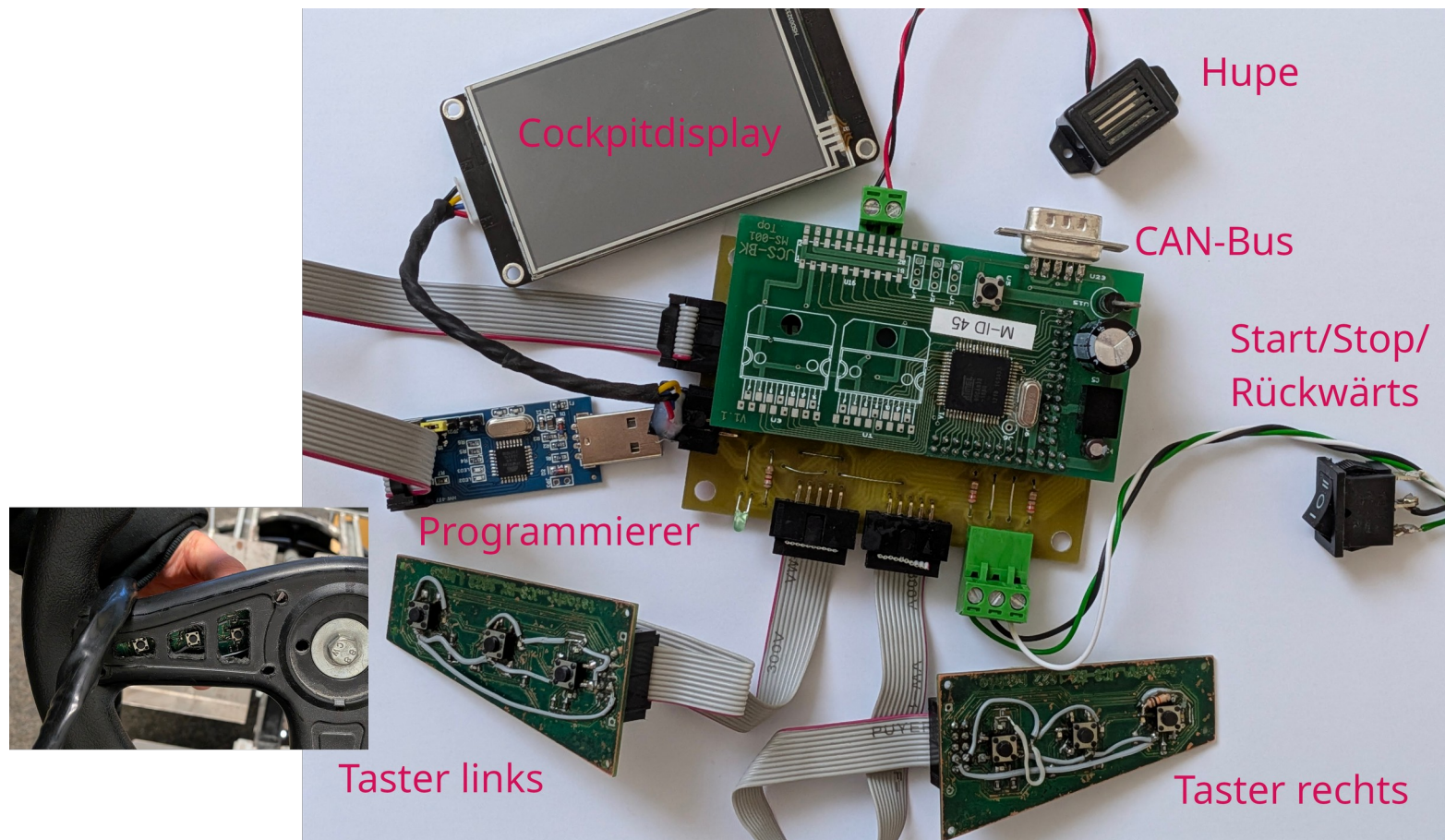


Abbildung 3: Cockpitplatine mit allen Geräten angeschlossen

Die untere Platine wurde in diesem Projekt erstellt. Die grüne LED zeigt an ob 5V anliegen. Dabei werden die Taster werden wie in dem Bild links in dem Lenkrad befestigt.

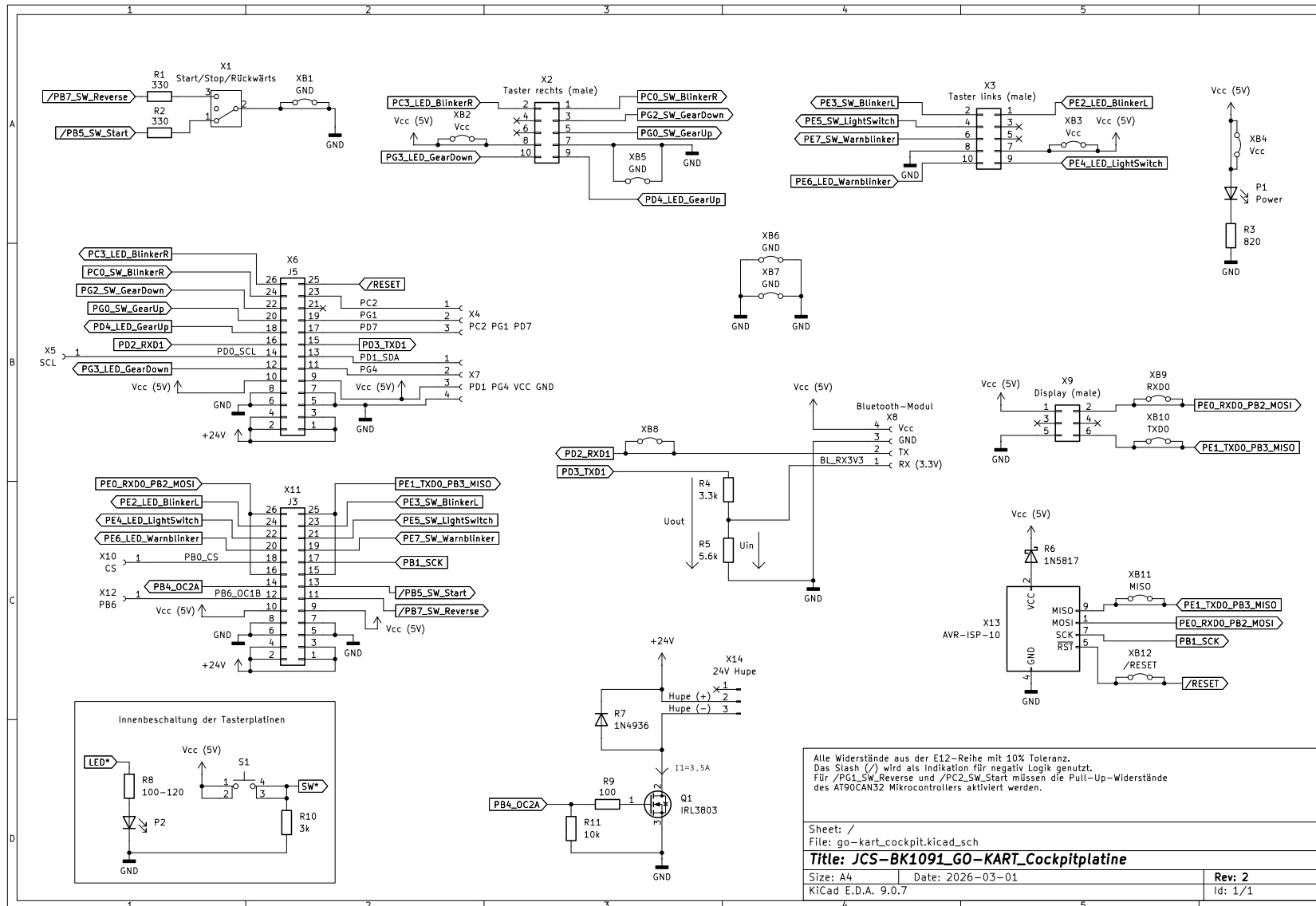


Abbildung 4: Schaltplan der Cockpitplatine Rev 2

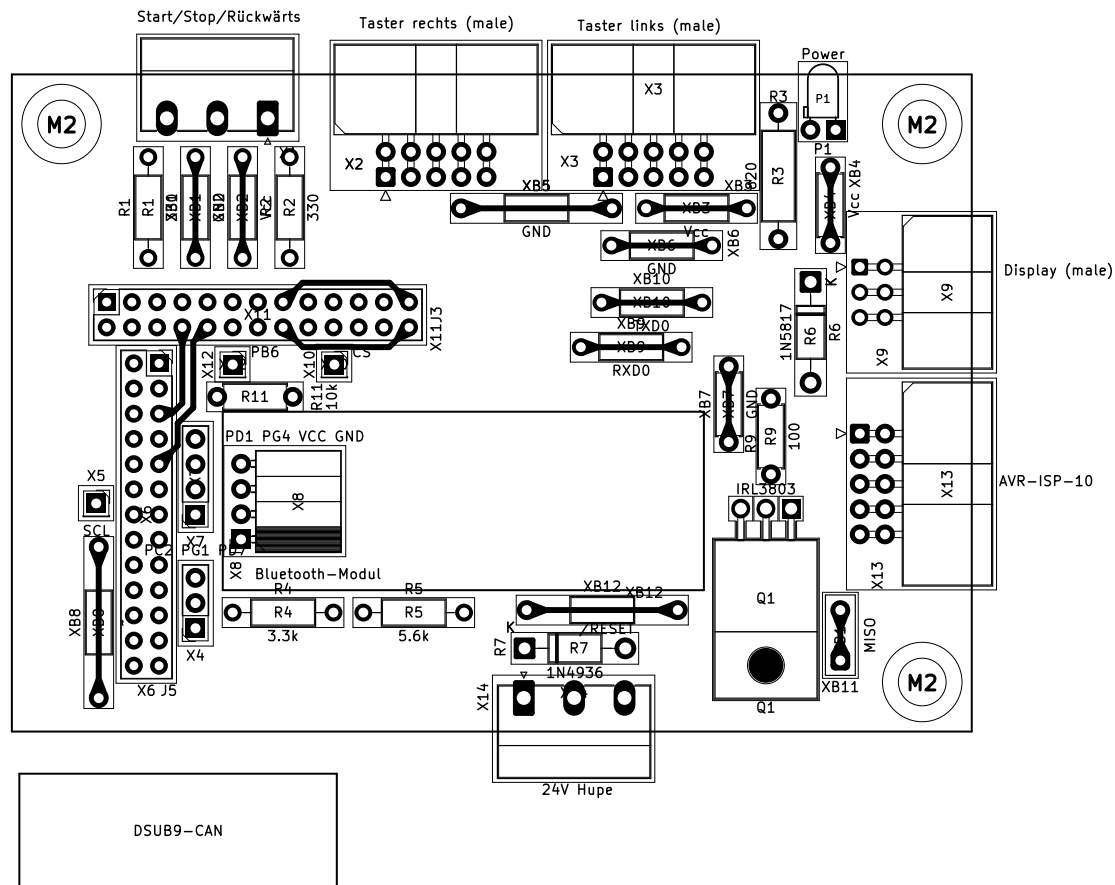


Abbildung 5: Bauteilansicht (top) der Cockpitplatine Rev 2

Der einzige Fehler welcher korrigiert werden musste, ist eine Brücke zwischen MOSI und RXD0 bzw. MISO und TXD0. Diese Änderung ist in diesem Schaltplan bereits korrigiert. Jeder Schaltplanausschnitt in dieser Dokumentation entspricht der Realität. Eine Besonderheit des Schaltplanes (siehe Abbildung 4, S. 10) ist die Verwendung von Labels, so sind zum Beispiel „Vcc (5V)“ der LED und alle weitere Anschlüsse mit der Bezeichnung „Vcc (5V)“ in Wirklichkeit miteinander Verbunden.

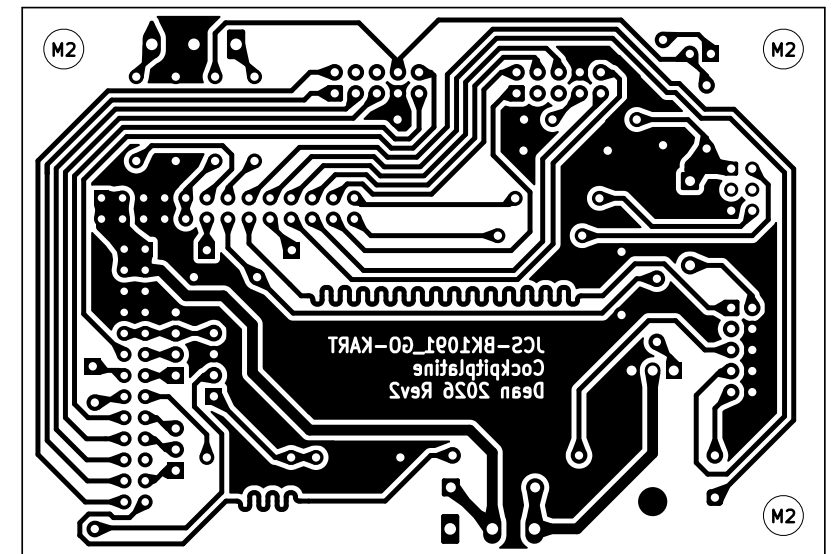


Abbildung 6: Leiterbahnen (bottom) der Cockpitplatine Rev 2

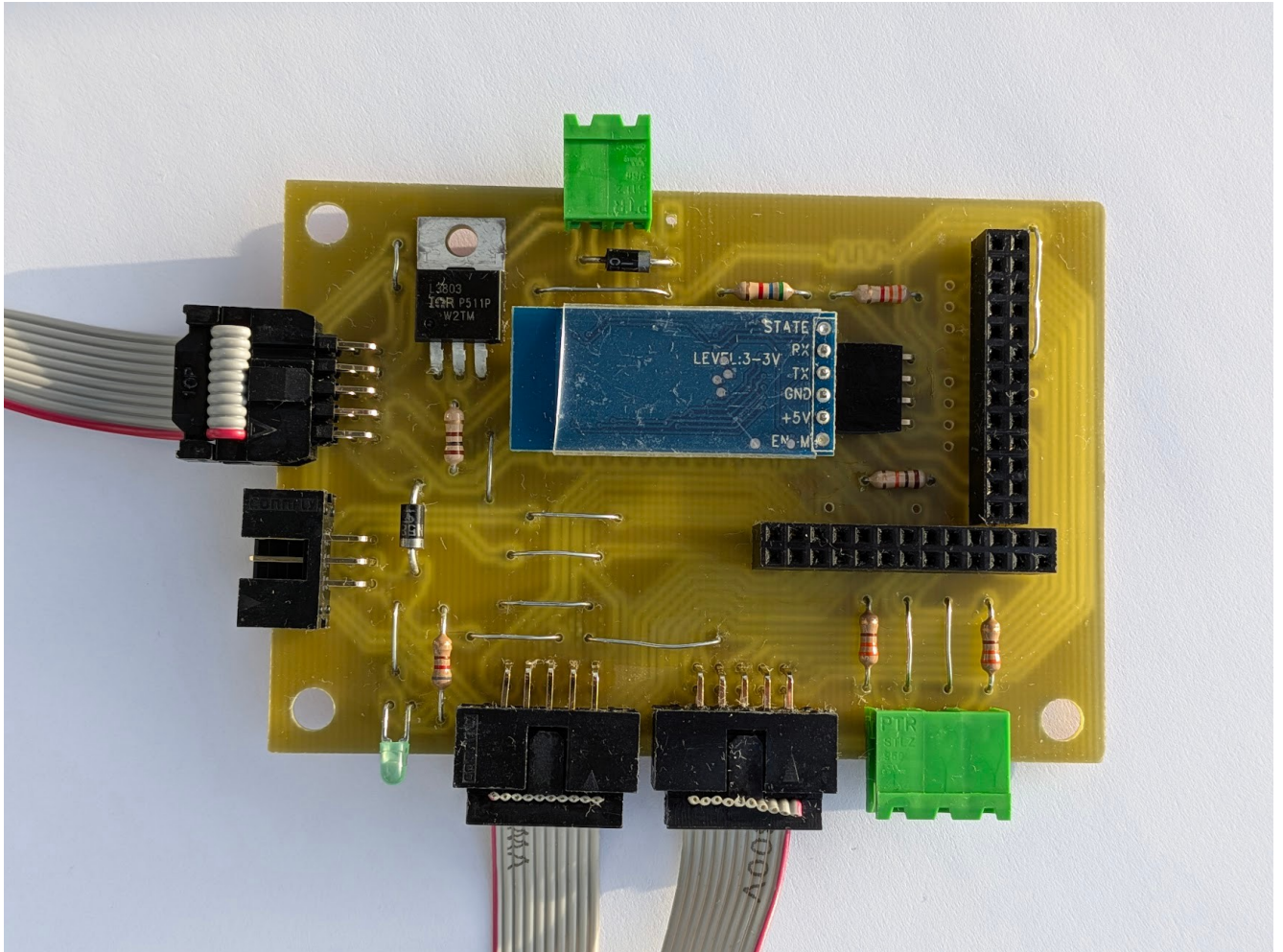


Abbildung 7: Bluetooth-Modul auf der Cockpitplatine

Name	Go-Kart_Cockpit
Passowort	1234
Baudrate	9600 bit pro Sekunde

Tabelle 3: Cockpit Bluetooth-Modul Einstellungen

Das Cockpit, sprich der AT90CAN32 Microcontroller des Cockpits kann über CAN-Bus mit den anderen Projekten und den Blinkern kommunizieren. Außerdem gibt es eine Ausgabe über Bluetooth. Wie man über Bluetooth sendet, wird später erklärt (siehe 4.2 UART Übertragung, S. 55).

2.4 Blinker-Module

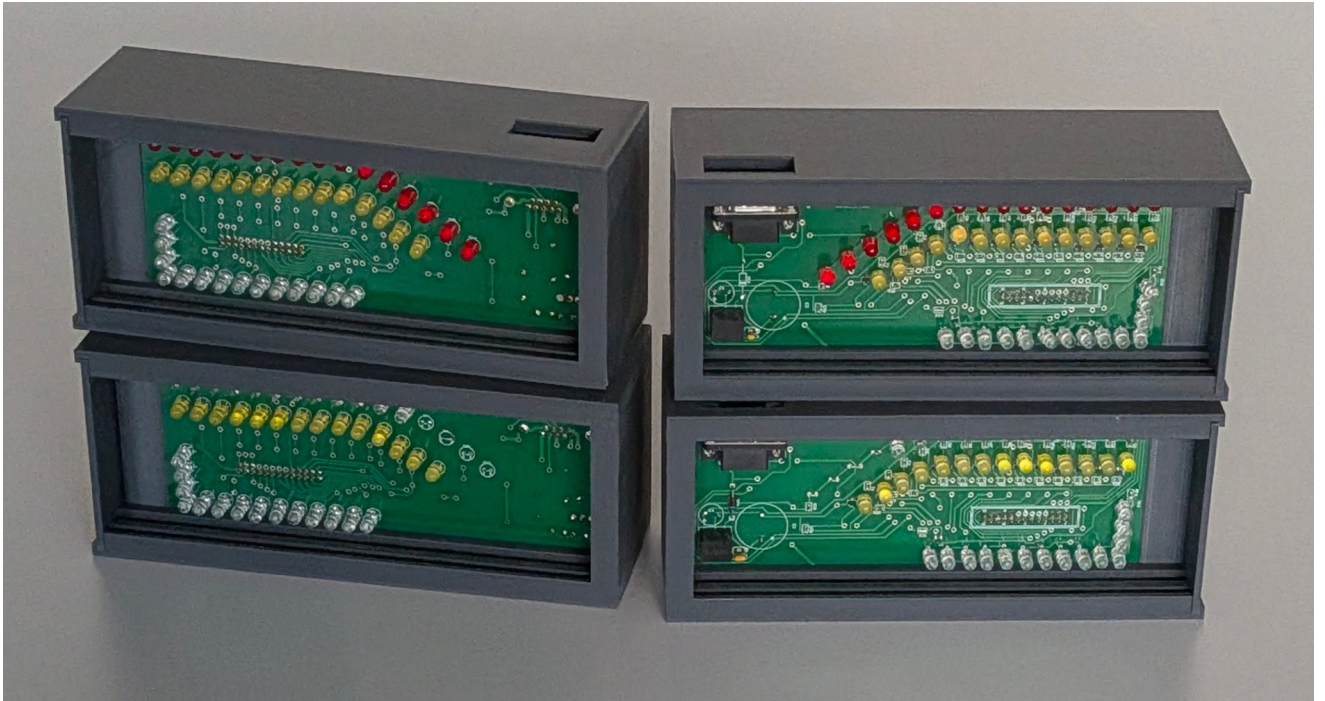


Abbildung 8: Alle vier Blinker-Module

Name	Go-Kart_Blinker
MAC-Adresse	00:21:13:01:54:0F
Passowort	1234
Baudrate	9600 bit pro Sekunde

Tabelle 4: Blinker Bluetooth-Modul Einstellungen

Diese Blinker-Module habe ich fertig erhalten. Die Platine wurde von Christoph Waßmuth in dem Projekt JCS-BK_1025 entwickelt.

Alle Blinker-Module wurden auf Funktion überprüft. Dabei haben alle funktioniert bis auf vorne links, wahrscheinlich hat dieser noch nicht die Brücke welche bei den anderen hinzugefügt wurde.

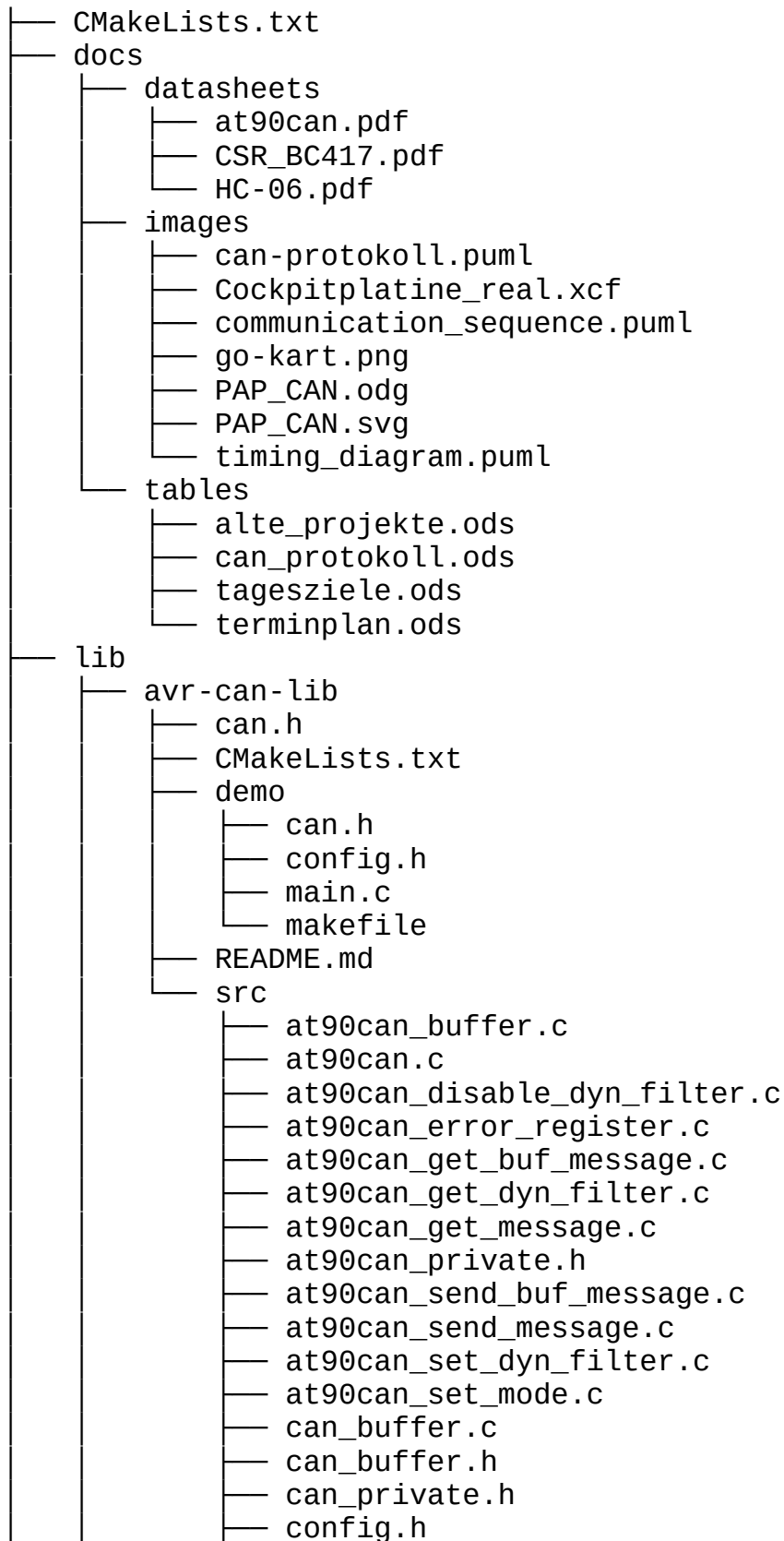
Jedes Blinker-Modul kann einzeln über CAN-Bus oder per Bluetooth auch mit Textbefehlen gesteuert werden. Wie man über Bluetooth sendet, wird später erklärt (siehe 4.2 UART Übertragung, S. 55). Durch sendet des Buchstaben „h“ erhält man eine Liste der möglichen Befehle. Alle vier Blinker-Module teilen sich ein Bluetooth-Modul.

Hilfe Ausgabe von dem Blinker-Modul hinten links:

```
info: JCS-BK_1091_GO-KART von Dean Schneider (GYT26)
info: Blinker-Modul hinten links mit CAN-ID: 0x00000601
info: State Nachrichten werden an 0x00000703 gesendet.
info: UART Befehle:
info: an='1', aus='2': Blinker
info: an='3', aus='4': Rueckfahrlicht
info: '5': Ruecklicht
info: '6': Bremslicht
info: '7': kein oberes Licht
```

2.5 Dateistruktur

Der Quellcode und weitere Dateien des Projektes sollten von dem Projektleiter bereitgestellt werden. Die Diagramme wurden mit PlantUML erstellt, die Rohdateien dazu sind in den .puml Dateien zu finden.



```

|
|
|   |— doxyfile
|   |— makefile
|   |— mcp2515_buffer.c
|   |— mcp2515.c
|   |— mcp2515_defs.h
|   |— mcp2515_error_register.c
|   |— mcp2515_get_dyn_filter.c
|   |— mcp2515_get_message.c
|   |— mcp2515_private.h
|   |— mcp2515_read_id.c
|   |— mcp2515_regdump.c
|   |— mcp2515_send_message.c
|   |— mcp2515_set_dyn_filter.c
|   |— mcp2515_set_mode.c
|   |— mcp2515_sleep.c
|   |— mcp2515_static_filter.c
|   |— mcp2515_write_id.c
|   |— sja1000_buffer.c
|   |— sja1000.c
|   |— sja1000_defs.h
|   |— sja1000_error_register.c
|   |— sja1000_get_message.c
|   |— sja1000_private.h
|   |— sja1000_send_message.c
|   |— sja1000_set_mode.c
|   |— spi.c
|   |— spi.h
|   |— utils.h
|
|   |— uart
|       |— CMakeLists.txt
|       |— include
|           |— uart_config.h
|           |— uart.h
|       |— src
|           |— uart.c
|
|— LICENSE
|— README.md
|— src
    |— blinker
        |— blinker_controller.c
        |— hal.h
        |— main.c
        |— uart_AM16M1.h
    |— cockpit
        |— cockpit_display.c
        |— cockpit_display.h
        |— hal.h
        |— main.c

```

2.6 CAN-Bus

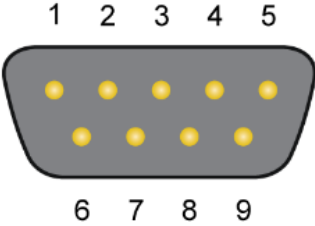
Pin	Assignment	D-Sub plug
1	CAN_V+ (optional)	
2	CAN_Low	
3	CAN_GND	
4	Not connected	
5	Not connected	
6	CAN_GND	
7	CAN_High	
8	Not connected	
9	CAN_V+ (optional)	

Abbildung 9: männlicher CAN D-Sub Anschluss nach CiA® 106
(Quelle: PCAN-USB Anleitung)

CAN-Bus Baudrate	125 kbit pro Sekunde
------------------	----------------------

Tabelle 5: CAN-Bus Einstellungen

2.7 Mikrocontroller

2.7.1 ATmega16M1

Für die Lichtanlage wird der ATmega16M1 Mikrocontroller verwendet.

Die wichtigsten Eigenschaften des ATmega16M1:

- 16 KB Flash, 1 KB SRAM, 512 byte EEPROM
- Hardware CAN-Controller
- Hardware UART mit LIN
- 2x 8-bit Timer und 1x 16-bit Timer für PWM
- Analog-Digital-Wandler

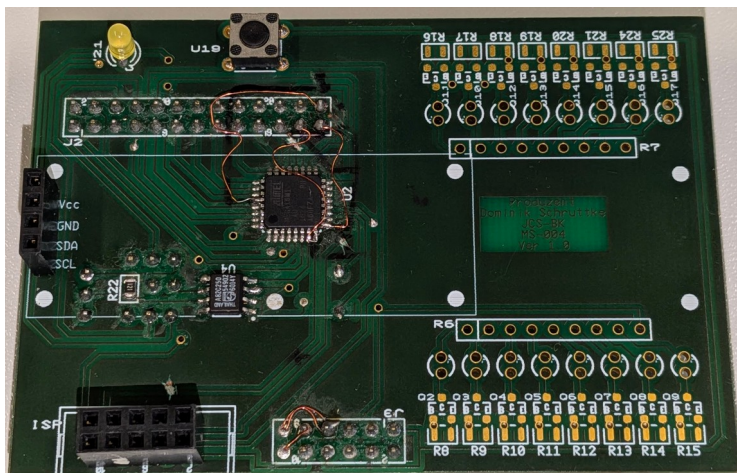


Abbildung 11: Oberseite des ATmega16M1

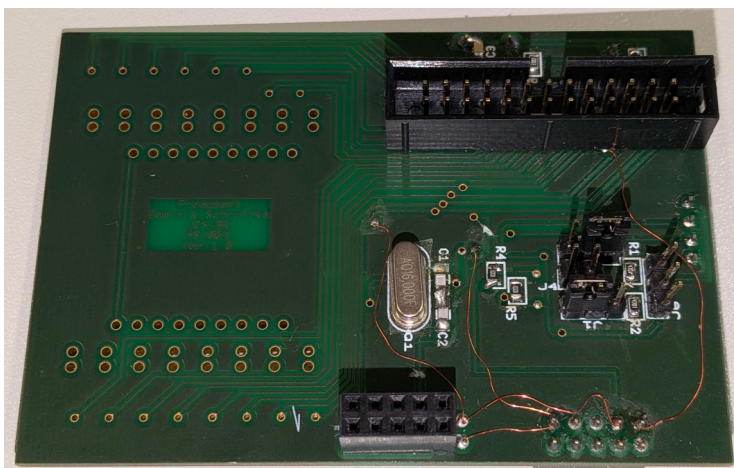


Abbildung 12: Unterseite des ATmega16M1

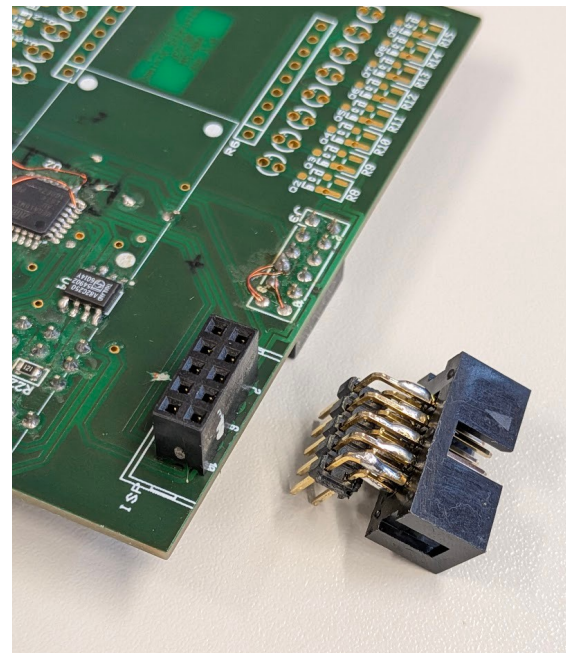


Abbildung 10: Stecker zum Programmieren des ATmega16M1

2.7.2 AT90CAN32

Die Platine mit dem AT90CAN32 Mikrocontroller wird für das Cockpit-Display und das Lenkrad verwendet. Da es die zentrale Steuereinheit für dieses Projekt ist, wird sie auch als Masterplatine bezeichnet. Das Projekt JCS-BK MS-001 enthält Schaltpläne zur Masterplatine. Pläne zur Addon-Platine, das heißt dort wo die serielle und ISP Verbindung angeschlossen wird, lassen sich unter JCS-BK MS-003 finden.

Die wichtigsten Vorteile des AT90CAN32 gegenüber dem ATmega16M1 für dieses Projekt sind:

- zwei USART-Schnittstellen gegenüber einer beim ATmega16M1
- stärkerer CAN-Controller mit mehr Funktionen und Message Objects:
 - AT90CAN32: 15 Full Message Objects
 - ATmega16M1: 6 Message Objects
- Hardware I2C-Controller

Der AT90CAN32 wurde explizit für seine zwei USART-Schnittstellen verwendet.

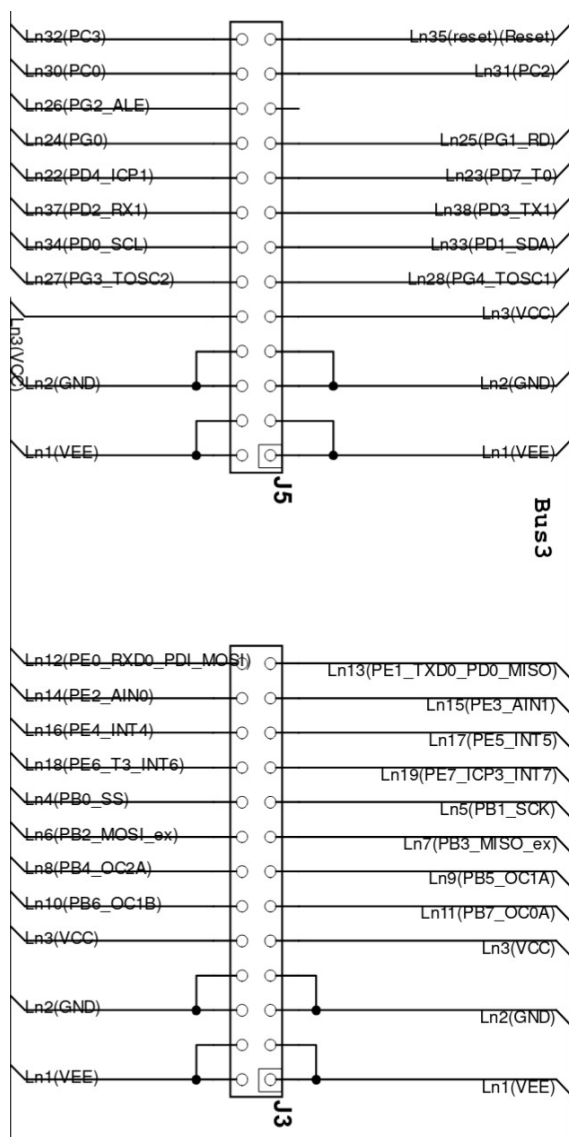


Abbildung 13: Anschlüsse des AT90CAN32 (Quelle: Schaltplanausschnitt von MS-001)

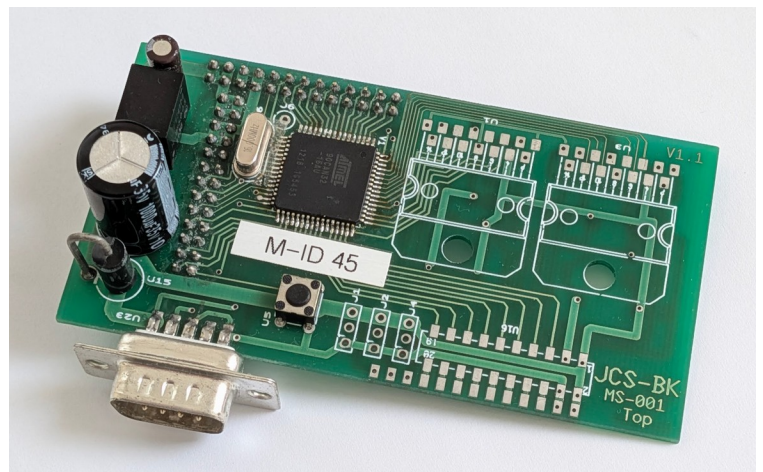


Abbildung 14: Masterplatine mit AT90CAN32 (MS-001)

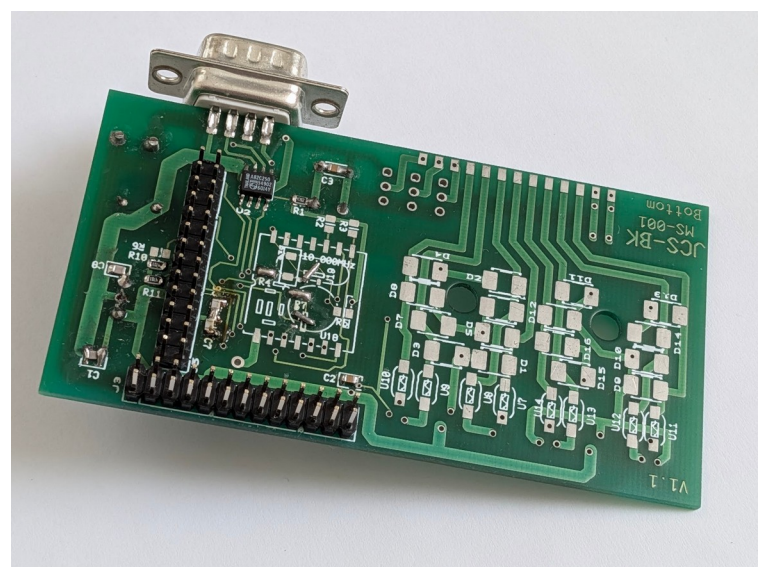


Abbildung 15: Masterplatine mit AT90CAN32 (MS-001)

3 Entwicklung

In diesem Kapitel wird der Prozess wie es Ergebnis kam dargestellt. Der aktuelle Stand des Projektes befindet sich im Kapitel 2 *Ergebnis* (Beginn S. 5).

3.1 Programmierer

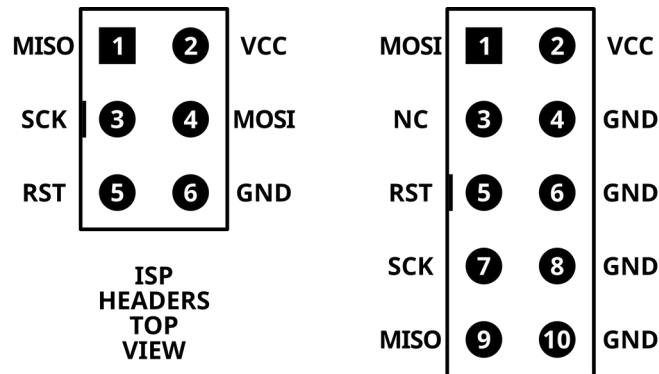


Abbildung 16: ISP Pinbelegung (Quelle: wikipedia.org von osiixy erstellt)

Ein Arduino besitzt einen Bootloader welcher den Eingang des UART in den Speicher schreibt, er programmiert sich also selbst. Allerdings besitzt der AVR MCU (AT90CAN32) von der Masterplatine keinen Bootloader, er wird über ISP (In-System Programming) programmiert.

Beim Kauf eines Programmierers ist darauf zu achten, dass er eine Open-Source Firmware hat, zum Beispiel USBasp.

Falls man nicht weiß welche Programmierer-Firmware man hat, kann man diese über die ID herausfinden:

```
$ lsusb
```

Zum Hochladen wird avrdude verwendet dasselbe Tool wie Microchip Studio verwendet. Als Erstes muss das Paket avrdude installiert werden. Dieses Paket ist nicht unbedingt überall vorhanden. Falls ein Paket nicht vorhanden ist kann man es auch immer über eine normale Intersuche finden und herunterladen (<https://github.com/avrdudes/avrdude/releases>). Zum Beispiel auf Arch mit:

```
$ sudo pacman -S avrdude
```

Anschließend kann man sich mit diesem Befehl alle voreingestellte Programmierer-Konfigurationen ausgeben lassen:

```
$ avrdude -c ?
```


3.2 Erste Schritte mit Masterplatine

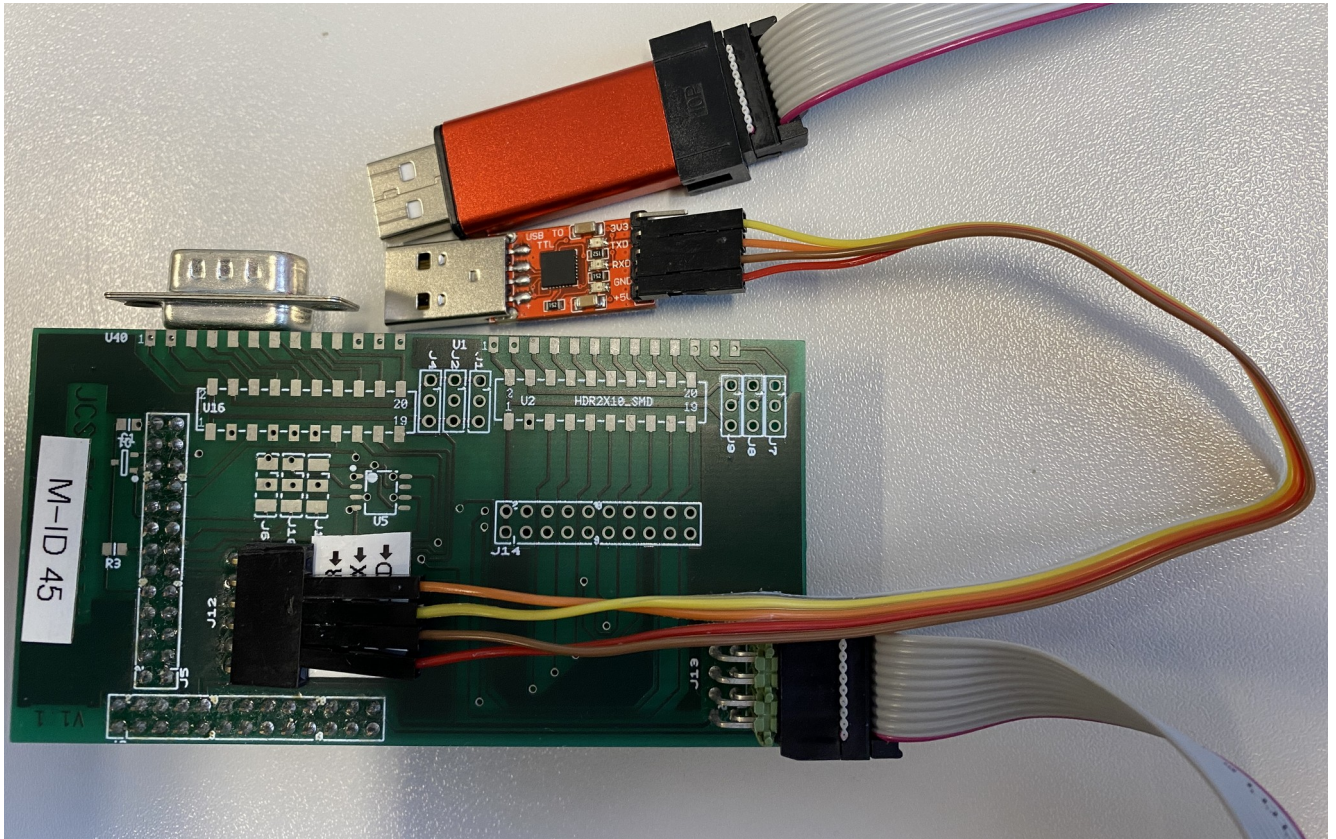


Abbildung 17: Masterplatine mit serieller und ISP Verbindung

Bevor man komplexe Programme programmiert sollte man erstmal die Programmier-Schnittstelle überprüfen. In diesem Beispiel wird TX zwischen High und Low gewechselt. Mein USB-zu-Seriell Gerät hat eine LED für RX, darüber kann ich den Spannungswechsel sehen. Kurz gesagt wird eine blinkende LED zum Feststellen der Funktion verwendet.

Warnung: In Abbildung 17 ist der Aufbau zu sehen. Es ist wichtig zu überprüfen ob der Programmierer eine Spannung liefert (z. B. 5V), denn dann sollte die Spannungsversorgung des USB-zu-Seriell Adapters von der Masterplatine getrennt werden. Oft ist dies nicht nötig, es reduziert jedoch das Risiko eines Schadens durch Ausgleich-Ströme.

Hier ist der Code zum Blinken lassen der RX LED (siehe 4.1 Kompilieren und Hochladen, S. 54):

```
#include <avr/io.h>
#include <util/delay.h>

static const uint8_t TXD1 = PD3;

int main(void) {
    DDRD = 1 << TXD1;

    while (1) {
        PORTD ^= 1 << TXD1;
        _delay_ms(500);
    }
}
```

Die Schritte zum hochladen werden in den Grundlagen erklärt (siehe 4.1 Kompilieren und Hochladen, S. 54).

3.3 Entwicklung mit HAL

Dieses Projekt verwendet für jeweils den Blinker und das Cockpit ein Hardware-Abstraktions-Layer (https://en.wikipedia.org/wiki/Hardware_abstraction). Dadurch wird der Code lesbarer und Fehler werden vermieden. Die Besonderheit ist sowohl der Port als auch der Bit/Pin werden in einem Define gespeichert und anschließend mit Makros wieder extrahiert. Der zuvor gezeigte Blinktest würde dann so in eine hal.h C-Datei geschrieben werden:

```
#ifndef HAL_H
#define HAL_H

#include <avr/io.h>

// Beispiel: PORTC PC6 => C, 6

#define UART1_TX D, 3

#define _SET2(port, bit) PORT##port |= (1 << bit)
#define _GET2(port, bit) ((PIN##port >> bit) & 1)
#define _RESET2(port, bit) PORT##port &= ~(1 << bit)
#define _TOGGLE2(port, bit) PORT##port ^= (1 << bit)
#define _SET_INPUT2(port, bit) DDR##port |= (1 << bit)
#define _SET_OUTPUT2(port, bit) DDR##port |= (1 << bit)

#define SET(port_comma_bit) _SET2(port_comma_bit)
#define GET(port_comma_bit) _GET2(port_comma_bit)
#define RESET(port_comma_bit) _RESET2(port_comma_bit)
#define TOGGLE(port_comma_bit) _TOGGLE2(port_comma_bit)
#define SET_INPUT(port_comma_bit) _SET_INPUT2(port_comma_bit)
#define SET_OUTPUT(port_comma_bit) _SET_OUTPUT2(port_comma_bit)

#define _P2(port, bit) (1 << bit)
#define _P(port_comma_bit) _P2(port_comma_bit)

void hal_init(void) {
    // set output pins
    DDRD = _P(UART1_TX);
}

#endif
```

Anschließend kann man Pins wie folgt ansteuern und auch auslesen (main.c):

```
#include <avr/io.h>
#include <util/delay.h>
#include "hal.h"

int main(void) {
    hal_init();

    while (1) {
        TOGGLE(UART1_TX);
        _delay_ms(500);
    }
}
```

3.4 Bluetooth-Modul

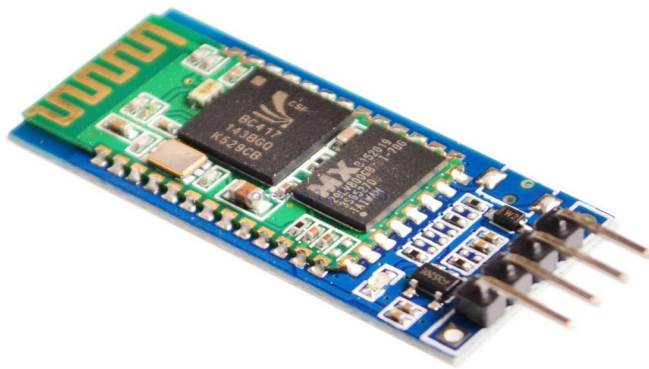


Abbildung 18: Bluetooth-Modul mit HC-06 Oberseite
(Quelle: roboter-bausatz.de)

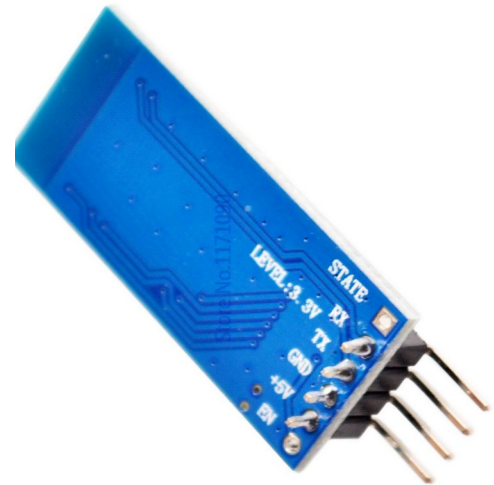


Abbildung 19: Bluetooth-Modul mit HC-06 Unterseite
(Quelle: roboter-bausatz.de)

In Abbildung 18 ist das verwendete Bluetooth-Modul zu sehen. Bei der grünen aufgelöteten Platine handelt es sich um den HC-06 mit dem CSR BC417BQN (Bluetooth Chip). Der HC-06 unterscheidet sich vom HC-05, indem er nur als Slave fungieren kann. Das bedeutet der HC-06 kann nicht ein Gerät (z. B. Laptop) suchen und sich mit ihm Verbinden. Zu erkennen ist der HC-06 daran, dass nur 4 Pins herausgeführt sind.

Die Pins TX und RX fungieren als UART Schnittstelle.

PIN	Beschreibung
RX (3, 3V)	Receive – Eingang UART des Moduls, benötigt einen Spannungsteiler
TX	Transmit – Ausgang UART des Moduls
GND	Verbindung zu GND
5V	+5V Versorgungsspannung

3.4.1 Test und konfigurieren mit USB-zu-Seriell-Konverter

Das Bluetooth-Modul wird mit dem USB-zu-Seriell-Konverter verbunden. Es sind keine Widerstände nötig da der Konverter mit 3,3 V arbeitet.

Bluetooth-Modul	USB-zu-Seriell-Konverter
RX	TX
TX	RX
GND	GND
5V	+5V

Die LED des Bluetooth-Moduls sollte blinken, sobald die Spannungsversorgung hergestellt wurde. Dies bedeutet das auf Verbindung gewartet wird.

Nun können die Befehle zum Konfigurieren über UART (siehe 4.2 UART Übertragung, S. 55) mit einer Baud-Rate von 9600 bps übertragen werden. Beim HC-06 kann kein „AT Command Mode“ aktiviert werden. Er befindet sich immer im Kommando-Modus, wenn keine Verbindung besteht was an dem Blinken der LED zu erkennen ist.

Befehl senden	Beispiel	Rückgabe vom Modul
AT		OK
AT+VERSION		OKlinvorV1.8
AT+NAME<Name>	AT+NAMEBlinker	OKsetname
AT+PIN<PIN>	AT+PIN1234	OKsetPIN

Es gibt weitere Befehle (siehe Datenblatt). So ist es zum Beispiel möglich die BAUD-Rate zu verändern, allerdings reicht 9600 bps völlig für den UART gebrauch aus und eine Änderung kann Projekt verwirren.

Schließlich kann man sich mit einem anderen Computer über Bluetooth Verbinden und seriell Daten an den Computer mit der UART Verbindung senden (siehe 4.2 UART Übertragung, S. 55).

3.4.2 Schaltung mit Mikrocontroller

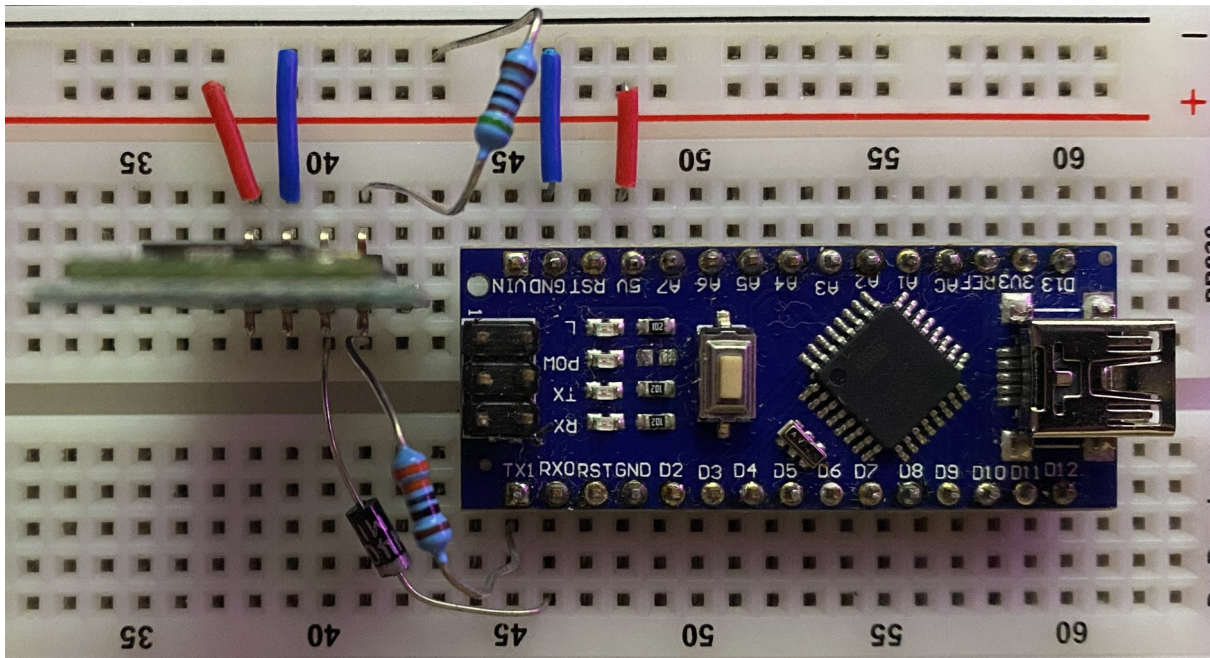
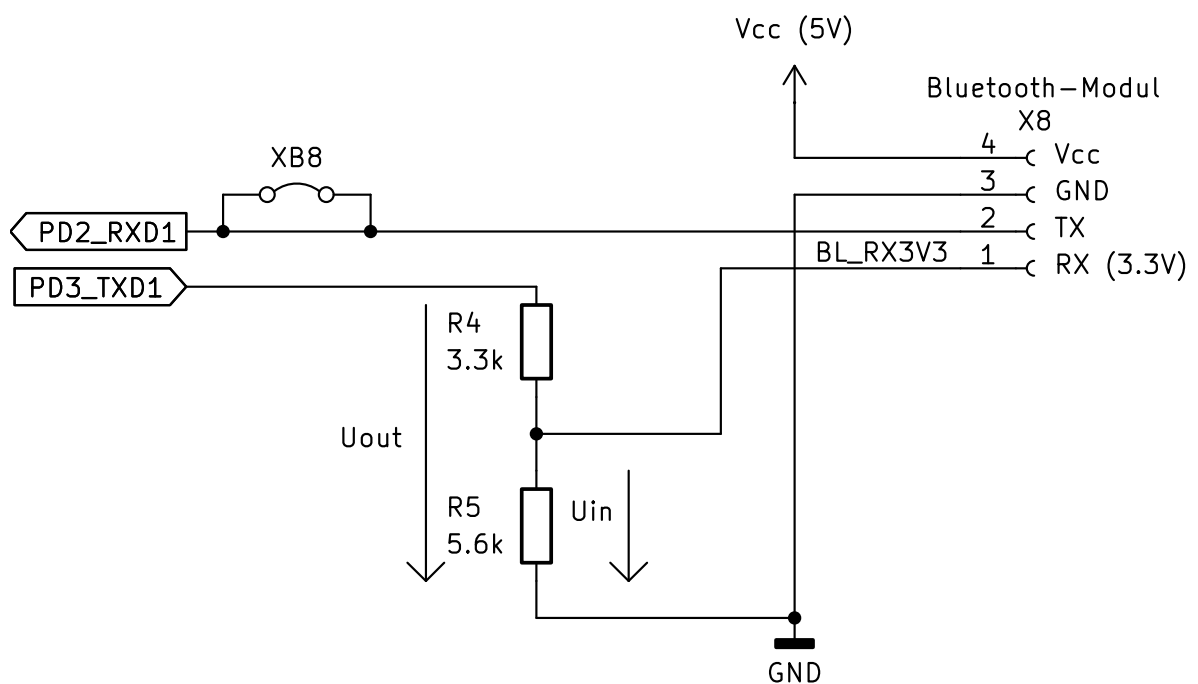


Abbildung 20: Bluetooth-Modul Schaltung mit Arduino



Schaltplanausschnitt 1: Anschluss des Bluetooth-Modul an die Masterplatine

Der TX Pin des Bluetooth-Moduls wird über eine Diode geleitet, da der MCU noch nicht initialisiert sein könnte und damit der RX des MCU High ist und TX Low, dadurch entsteht ein Kurzschluss. Der TX Pin des MCU wird über einen Spannungsteiler angeschlossen da der MCU mit 5V und das Modul mit 3.3 V arbeitet. Im Folgenden wird der Spannungsteiler dimensioniert.

Die Eingangsspannungen beziehen sich auf den CSR BC417BQN.

$U_{out} \in [4,6 \text{ V}; 5,2 \text{ V}]$	Ausgangsspannung vom MCU
$U_{in} \in [2,2 \text{ V}; 4,2 \text{ V}]$	Eingangsspannung
$U_{in_{max}} = 5,6 \text{ V}$	Absolute Maximum

$$R_1 = 3,3 \text{ k}\Omega \pm 10 \%$$

$$R_2 = 5,6 \text{ k}\Omega \pm 10 \%$$

$$U_{in} = U_{out} \cdot \frac{R_1}{R_2}$$

$$\hat{U}_{in} = 5,2 \text{ V} \cdot \frac{3,3 \text{ k}\Omega \cdot 1,1}{5,6 \text{ k}\Omega \cdot 0,9} \approx 3,75 \text{ V}$$

$$\bar{U}_{in} = 4,8 \text{ V} \cdot \frac{3,3 \text{ k}\Omega}{5,6 \text{ k}\Omega} \approx 2,83 \text{ V}$$

$$\tilde{U}_{in} = 4,6 \text{ V} \cdot \frac{3,3 \text{ k}\Omega \cdot 0,9}{5,6 \text{ k}\Omega \cdot 1,1} \approx 2,22 \text{ V}$$

Schließlich kann man die UART-Ausgabe über Bluetooth auslesen und Daten an den MCU über UART senden (siehe 4.2 UART Übertragung, S. 55).

Warnung: Hochladen über den Bootloader wird nicht funktionieren da ein Reset benötigt wird.

3.5 Blinker-Modul



Abbildung 21: Blinker auf Go-Kart positioniert

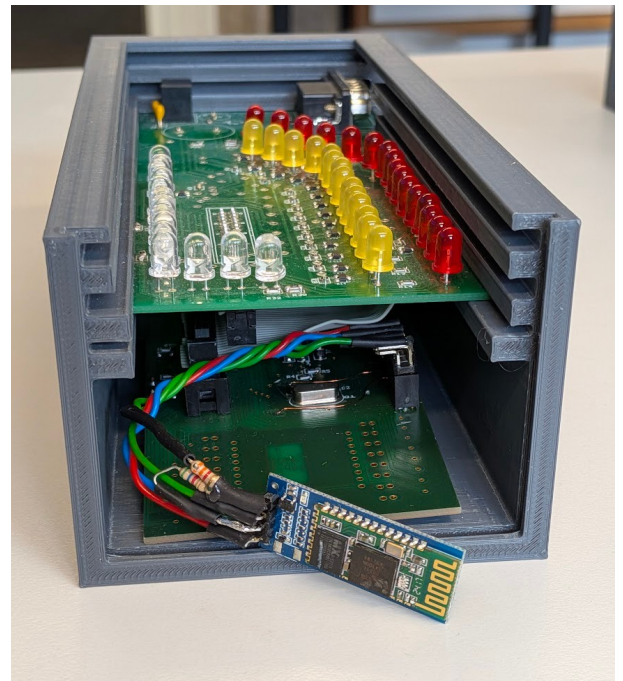


Abbildung 22: Innenleben des Blinker-Moduls hinten rechts

In Abbildung 22 sieht man das Blinker-Modul mit der Microcontroller-Platine und wie das Bluetooth-Modul angeschlossen. Bei dem Bluetooth-Modul ist zu beachten, dass der Winkel nach innen zeigt.

3.5.1 Berechnung des Timers

Nach der Norm ECE 6.4 muss die Lampe für die Fahrtrichtungsanzeige mit einer Frequenz von 1,5 Hz den Zustand wechseln, mit einer Tolleranz von 0,5 Hz. Angeschalten gilt die Lampe, wenn 95% der LEDs angeschalten sind, dies ist nur der Fall, wenn alle 14 LEDs leuchten.

$$n = 14 \text{ LEDs}$$

$$f = 1,5 \text{ Hz} \pm 0,5 \text{ Hz}$$

$$T_{Hmin} = 300 \text{ ms}$$

Aufgrund das der Blinker animiert ist und sonst zu schnell durchlaufen würde, wurde eine etwas geringere Frequenz gewählt.

$$f = 1,1 \text{ Hz} \Rightarrow T = \frac{1}{f} = \frac{1}{1,1 \text{ Hz}} = 909 \text{ ms}$$

Das folgende Signalzeitdiagramm zeigt den zeitlichen Verlauf der LEDs An bzw. Auszustände. Bei der Programmierung wird ein Timer mit Interrupt verwendet. ISR steht dabei für die Interrupt-Service-Routine. Man kann auch sagen jedes mal wenn ISR auf 1 geht findet ein Wechsel statt, sei es eine LED die oder aus geht oder einfach nichts tun.

Gesucht bei der Berechnung ist die Frequenz für den Timer.

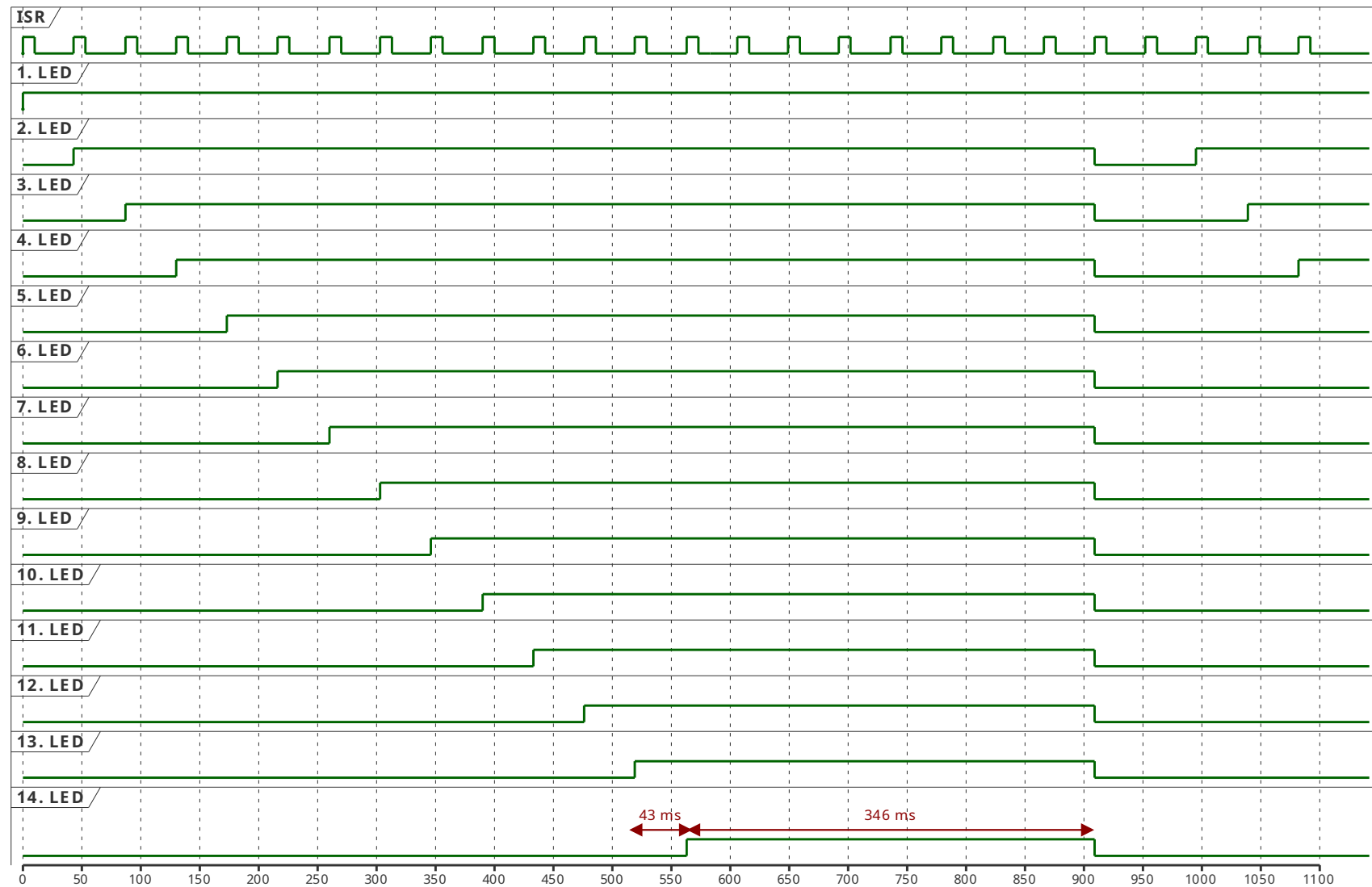


Abbildung 23: Signalzeitdiagramm für Blinker-LEDs

Die in Abbildung 23 zusehenden Zeiten ergeben sich für 43 ms aus $1/f_{\text{Timer}}$ und 346 ms ist T_H .

$$\begin{aligned}\frac{T_{Hmin}}{T} &= \frac{n_H}{n + n_H} \\ \frac{300 \text{ ms}}{909 \text{ ms}} &= 0,33 \\ \frac{n_H}{n + n_H} &= 0,33 & | \cdot (n + n_H) \\ n_H &= 0,33 \cdot (n + n_H) \\ n_H &= 0,33 \cdot n + 0,33 \cdot n_H & | - 0,33 \cdot n_H \\ 0,67 \cdot n_H &= 0,33 \cdot n & | : 0,67 \\ n_H &= \frac{0,33}{0,67} \cdot n \\ n_H &= \frac{0,33}{0,67} \cdot 14 \approx 7 \\ n_{ges} &= 14 + 7 = 21\end{aligned}$$

Um die Anzahl der benötigten Aufrufe n_{ges} der Interrupt-Service-Routine zu finden, braucht man die Anzahl der Aufrufe in dem alle LEDs angeschalten sein sollen n_H . Diese wurden durch aufstellen eines Verhältnisses gefunden.

$$f_{\text{Timer}} = f \cdot n_{ges} = 1,1 \text{ Hz} \cdot 21 = 23,1 \text{ Hz}$$

Es steht nur noch der 8-Bit Timer0 zur Verfügung, da der 16-Bit Timer1 für das PWM verwendet wird. Mit 8-Bit kann man über die Hardware höchstens eine 255er Teilung erreichen, das reicht nicht für 23,1 Hz, daher muss zusätzlich eine Software Teilung stattfinden. Timer0 wird ebenfalls für die intervale Sendung von CAN-Nachrichten verwendet. Schließlich muss man Werte finden wo ein möglichst geringer Rundungsfehler auftritt.

$$\begin{aligned}f_{clk} &= 16 \text{ MHz} \\ f_{pre} &= \frac{f_{clk}}{1024} = \frac{16 \text{ MHz}}{1024} = 15\,625 \text{ Hz} \\ f_{\text{Timer0}} &= \frac{15\,625 \text{ Hz}}{84} = 186 \text{ Hz} \\ f_{soft} &= \frac{186 \text{ Hz}}{8} = 23,25 \text{ Hz} \approx f_{\text{Timer}} = 23,1 \text{ Hz}\end{aligned}$$

So würde man den Timer mit den Registern Konfigurieren:

```
TCCR0A = 1 << WGM01; // reset Timer-Wert wenn OCR0A gleich ist
TCCR0B = 1 << CS02 | 1 << CS00;
OCR0A = 84;
```

Da die Interval-Nachrichten mit 1 Hz versendet wird, wäre die Initialisierung davon wie folgt:

```
OCR0B = 186;
```

Die Interrupt-Service-Routine sieht verkürzt so aus:

```
ISR(TIMER0_COMPA_vect) {
    // timer frequency / SOFT_PRESCALER = frequency
    static const uint8_t SOFT_PRESCALER = 8; // about 23.25 Hz

    bc_interrupt_count++;

    if (bc_interrupt_count != SOFT_PRESCALER) {
        return;
    }

    bc_interrupt_count = 0;

    switch (bc_current_stage) {
    case 0:
        SET(LED_BLINKER_0);
        break;
    case 1:
        SET(LED_BLINKER_1);
        break;
    ...
    case 13:
        SET(LED_BLINKER_13);
        break;
    }

    if (bc_current_stage == 14 + (7 - 1)) {
        reset_blinker_leds();
        RESET(LED_BLINKER_0);
    }

    bc_current_stage++;

    // 14 LEDs, 11 iterations with all enabled
    if (bc_current_stage >= 14 + 7) {
        bc_current_stage = 0;
    }
}
```

3.6 CAN-Bus

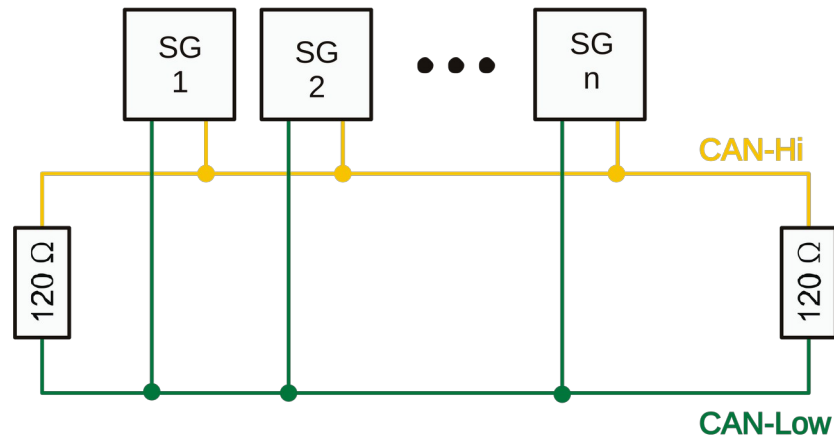


Abbildung 24: CAN bus elektrische Beispiel Topologie mit Abschlusswiderständen (Quelle: Stefan-Xp Wikipedia Nutzer)

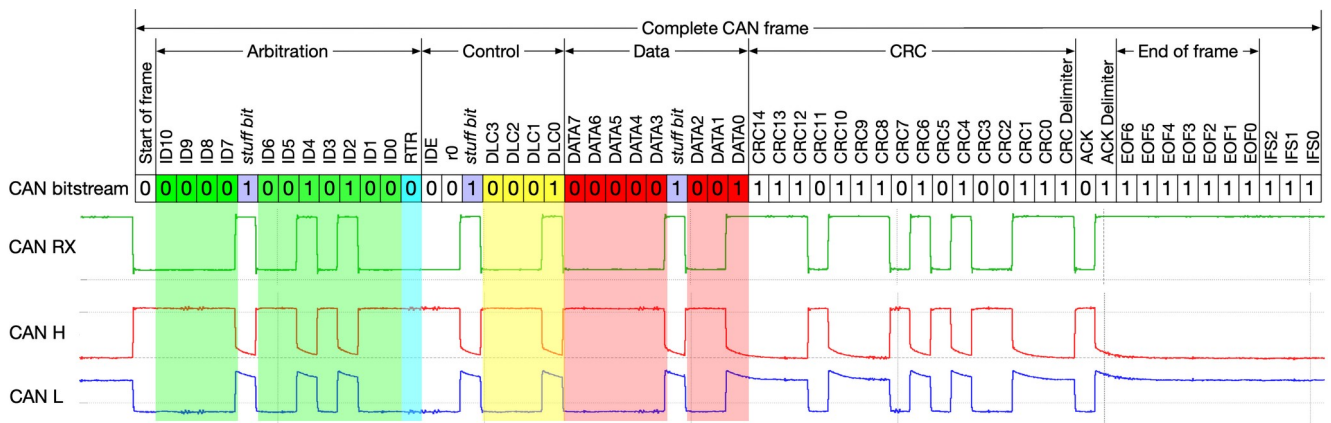


Abbildung 25: komplettes CAN frame Beispiel (Quelle: Canis Automotive Labs Ltd)

Der CAN-Bus (Controller Area Network) ist ein serielles Kommunikationssystem, das vor allem in Fahrzeugen sowie in Industrie- und Automatisierungssystemen eingesetzt wird. Mehrere Steuergeräte, sogenannte Nodes, sind dabei parallel an eine gemeinsame Busleitung angeschlossen und können über diese miteinander Daten austauschen. Die Kommunikation erfolgt über zwei Leitungen, CAN-High und CAN-Low, auf denen ein differenzielles Signal übertragen wird. Dadurch ist das System relativ unempfindlich gegenüber elektromagnetischen Störungen.

Damit die Signalübertragung stabil funktioniert, werden an beiden Enden der Busleitung Abschlusswiderstände eingesetzt. Diese haben typischerweise einen Widerstandswert von 120 Ω und verhindern Signalreflexionen auf der Leitung, die sonst zu Übertragungsfehlern führen könnten. In Abbildung 24 sind die beiden Abschlusswiderstände sowie mehrere angeschlossene Nodes dargestellt, die über CAN-High und CAN-Low mit dem Bus verbunden sind. Abzweigungen können über längere Distanz zu Problemen führen.

Die Kommunikation auf dem CAN-Bus erfolgt über sogenannte CAN-Frames, die die eigentlichen Daten übertragen. Ein Frame besteht aus mehreren Feldern, unter anderem einer Identifier-ID, die die Priorität der Nachricht festlegt, dem Datenfeld, in dem die Nutzdaten übertragen werden, sowie einer CRC-Prüfsumme, mit der Übertragungsfehler erkannt werden können. Abbildung 25 zeigt beispielhaft den Aufbau eines vollständigen CAN-Frames mit ID-, Daten- und CRC-Bereich.

3.6.1 Anschluss

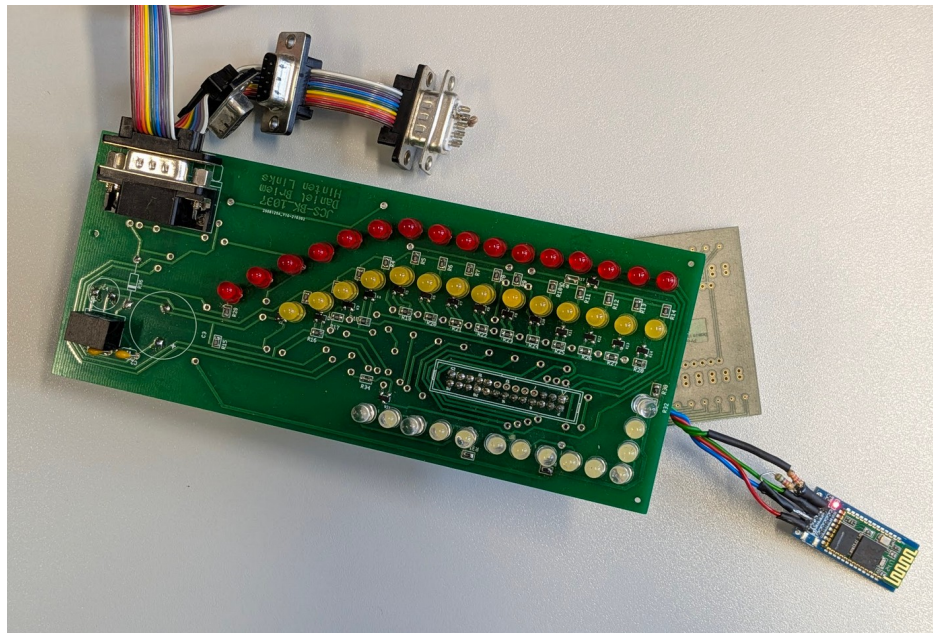


Abbildung 26: Blinker-Modul mit D-Sub Kabel

Pin	Assignment	D-Sub plug
1	CAN_V+ (optional)	
2	CAN_Low	
3	CAN_GND	
4	Not connected	
5	Not connected	
6	CAN_GND	
7	CAN_High	
8	Not connected	
9	CAN_V+ (optional)	

Abbildung 27: männlicher CAN D-Sub Anschluss nach CiA® 106
(Quelle: PCAN-USB Anleitung)

CAN-Bus wird mit einem D-Sub 9 Stecker bzw. Kabel angeschlossen. Dieses Kabel liefert ebenfalls eine 0V und 24V Leitung. In Abbildung 1 sieht man das angeschlossene CAN-Bus-Kabel.

3.6.2 Beispiel-Programm

Dieses Beispiel-Programm stellt alle benötigten CAN-Bus funktionen dar. Die erhaltene CAN-Nachricht an die ID=00001234h wird wieder zurückgeschickt, also ein Echo.

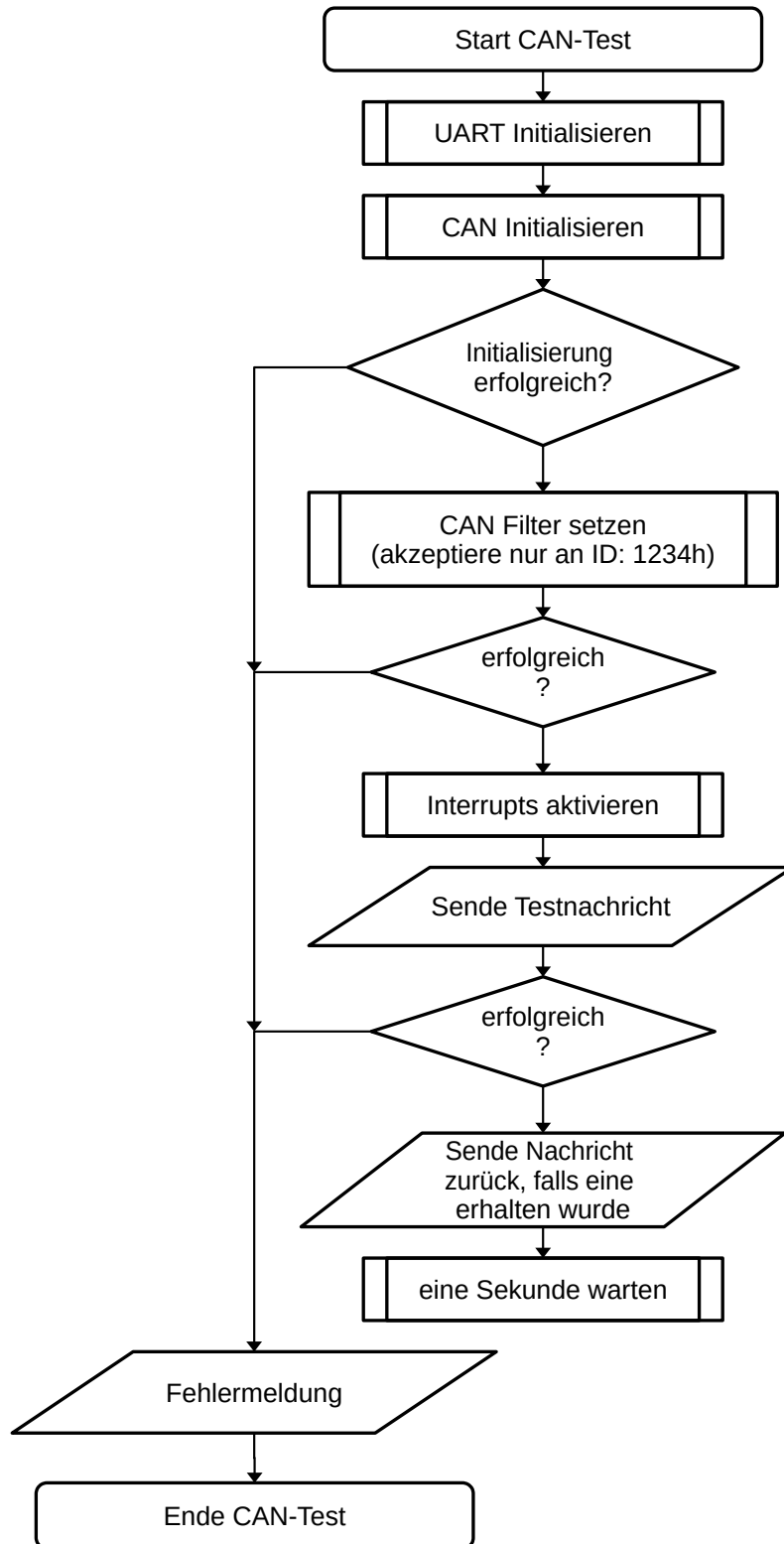


Abbildung 28: PAP zu dem CAN Testprogramm

```

#include "can.h"
#include "usart.h"
#include <avr/interrupt.h>
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    uart0_init(BAUD_CALC(9600UL));

    if (!can_init(BITRATE_125_KBPS)) {
        uart0_puts("error: while can initialization\r\n");
        while (true);
    }

    can_filter_t can_filter = {
        .id = 0x00001234,
        .mask = 0x1FFFFFFF,
        .flags.extended = 0x3, // filter with extended id
        .flags.rtr = 0x2,      // receive both RTR and normal
    };

    if (!can_set_filter(0, &can_filter)) {
        uart0_puts("error: while setting can message filters\r\n");
        while (true);
    }

    const can_t msg = {
        .id = 0x12340000,
        .flags.extended = true,
        .flags.rtr = false,
        .length = 1,
        .data = {0xAA},
    };

    sei();

    while (1) {
        if (can_send_message(&msg) == 0) {
            uart0_puts("error: can_send_message\r\n");
            continue;
        }

        can_t received_command;

        if (can_get_message(&received_command)) {
            received_command.id = 0x12340000;
            can_send_message(&received_command); // echo for testing
        }

        _delay_ms(1000);
    }
}

```

3.7 Erstes Display (Tacho)

Eigentlich sollte das NX8048K070_011C Display von Nextion verwendet werden. Allerdings war ich direkt zu Beginn des Projektes unaufmerksam und habe aus versehen an das 5V Display 14V angeschlossen, das Display leuchtete bis die Spannungsversorgung entfernt wurde. Der Spannungsregler des Displays ist beschädigt und eventuell auch weitere Bauteile. Diese müssten ausgetauscht werden, wobei sich die Frage stellt, ob sich der Zeitaufwand lohnt, insbesondere da es sich um SMD Bauteile handelt und man nicht eindeutig die Bauteilbezeichnung lesen kann.



Abbildung 29: Nextion Display (NX8048K070_011C)

V6H	XC6702D771PR-G	Torex Semiconductor SOT-89-5		Linear voltage regulator IC LDO, 7.7V±1%, 300mA, +CE
V6H	XC6702D771ER-G	Torex Semiconductor USP-6C		Linear voltage regulator IC LDO, 7.7V±1%, 300mA, +CE

Abbildung 30: Ersatz Bauteil suche für Spannungsregler (Quelle: <https://smd.yooneed.one>)

Nun muss das vorherige Display, welches allerdings von JCS-BK_1046 bereits gestaltet und eingestellt wurde, verwendet werden.

3.8 Cockpit-Display (Tacho)



Abbildung 31: Cockpit-Display (NX4024K032)

Das Display ist von Nextion und wird mit der Nextion typischen Software programmiert/eingestellt. Auf dem Glass befindet sich ein Riss von einem Vorgänger. Christian Barth (JCS-BK_1046) hat das Display gestaltet. Um die Symbole oder Zahlen zu ändern werden Befehle seriell mit einer Baudrate von 9600 bit/s gesendet (siehe 4.2.2 Rohdaten übertragen, S. 55).

3.8.1 Befehle

Funktion	ID	Wertebereich			
linker Blinker	bL.pic	ausgeschalten	0	angeschalten	1
rechter Blinker	bR.pic	ausgeschalten	2	angeschalten	3
Warnblinker	warnB.pic	ausgeschalten	15	angeschalten	16
Scheinwerfer	light.pic	Abblendlicht	17	Fernlicht	18
		Standlicht	19		
Drehzahl (Umdrehungen pro Minute)	rpm1.pic	Hunderter	4 – 13	Index 4 entspricht 0 und Index 13 entspricht 9	
	rpm2.pic	Zehner			
	rpm3.pic	Einer			
Geschwindigkeit (km/h)	speed1.pic	Zehner			
	speed2.pic	Einer			
gefahrene Distanz	dis1.pic	Hunderter			
	dis2.pic	Zehner			
	dis3.pic	Einer			
eingelegter Gang	gear.pic	Neutralgang	14	erster Gang	5
		zweiter Gang	6	dritter Gang	7
Batterie-Zustand	battery.pic	leer	20	1/4	21
		2/4	22	3/4	23
		4/4	24	wird geladen	25

Funktion	ID	Wertebereich
Gaspedal	throttle.val	0 – 100
Bremspedal	brake.val	100 – 0

Tabelle 6: Befehle für das Cockpit-Display

Alle angegebene Befehle bis auf „gefahrne Distanz“ wurde mit einem Testprogramm überprüft.

So kann man zum Beispiel mit dieser Nachricht an das Display den linken Blinker einschalten:

```
$ stty -F /dev/ttyUSB0 9600
$ printf 'bL.pic=1\xFF\xFF\xFF' > /dev/ttyUSB0
```

Erklärung dieser Befehle lässt sich unter *4.2.2 Rohdaten übertragen* (siehe S. 55) finden.

Mit AVR-C könnte das so realisiert werden:

```
uart_puts("bL.pic=1\xFF\xFF\xFF");
```

3.8.2 Beispiel-Programm

Dieses Beispiel-Programm lässt in Zeitabständen verschiedene Symbole und Zahlen auf dem Cockpit-Display anzeigen. Die Übertragung findet über UART statt. Um dieses Programm zu benutzen muss die Bibliothek-Datei „cockpit_display.h“ im selben Ordner wie die Main-Datei liegen. Die Bibliothek verwendet STB-Library-Stil, deswegen muss „COCKPIT_DISPLAY_IMPLEMENTATION“ vorher definiert werden.

In diesem Projekt wird die Bibliothek *AVR-UART-lib* von jnk0le (<https://github.com/jnk0le/AVR-UART-lib>) verwendet.

```
#include "usart.h"
#include <util/delay.h>

#include "cockpit_display.h"
#include "cockpit_display.c"

int main(void) {
    uart1_init(BAUD_CALC(9600UL));

    cd_set_rpm(789);
    cd_set_speed(12);
    cd_set_throttle(80);
    cd_set_brake(60);

    while (1) {
        cd_set_blinker_left(true);
        _delay_ms(200);
        cd_set_blinker_left(false);
        _delay_ms(200);

        cd_set_light(LIGHT_PARKING);
        _delay_ms(700);
        cd_set_light(LIGHT_LOW_BEAM);
        _delay_ms(700);
        cd_set_light(LIGHT_HIGH_BEAM);
        _delay_ms(700);
    }
}
```


3.8.3 Bibliothek Interface

```
#ifndef COCKPIT_DISPLAY_H
#define COCKPIT_DISPLAY_H

#include "usart.h"
#include <stdbool.h>
#include <stdint.h>

// these macros are used to choose between usart0 or usart1
#define CD_UART_PUTC uart0_putc
#define CD_UART_PUTS uart0_puts

typedef enum {
    GEAR_NEUTRAL,
    GEAR_FIRST,
    GEAR_SECOND,
    GEAR_THIRD,
    GEAR_STATE_COUNT,
} GearState;

typedef enum {
    LIGHT_OFF,
    LIGHT_DAY,
    LIGHT_LOW_BEAM,
    LIGHT_HIGH_BEAM,
    LIGHT_STATE_COUNT,
} LightState;

typedef enum {
    BATTERY_EMPTY,
    BATTERY_ONE_FOURTH,
    BATTERY_TWO_FOURTH,
    BATTERY_THREE_FOURTH,
    BATTERY_FOUR_FOURTH,
    BATTERY_CHARGING,
} BatteryState;

// Not implemented for now
// void cd_set_distance(uint8_t x);

void cd_set_blinker_left(bool enable);
void cd_set_blinker_right(bool enable);
void cd_set_warnblinker(bool enable);
void cd_set_light(LightState light_state);
void cd_set_rpm(uint16_t rpm);
/// @param speed_kmh range in [0; 99]
void cd_set_speed(uint8_t speed_kmh);
void cd_set_gear(GearState gear_state);
void cd_set_battery(BatteryState battery_state);
/// @param percent range in [0; 100]
void cd_set_throttle(uint8_t percent);
/// @param percent range in [0; 100]
void cd_set_brake(uint8_t percent);
#endif
```

3.9 Huppe

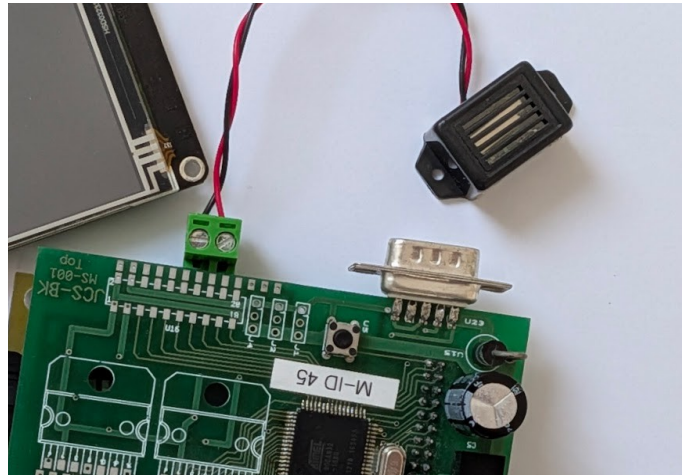


Abbildung 32: Miniatur Summer 24V (EMS-24L)

Da in meinem Pflichtenheft eine Hupe aufgelistet ist, muss diese auch eingebaut werden. Diese habe ich selber Bezahlt als kleine Entschädigung für das kaputte Display.

3.9.1 Erster Transistor

Figure 3. DC current gain (NPN)

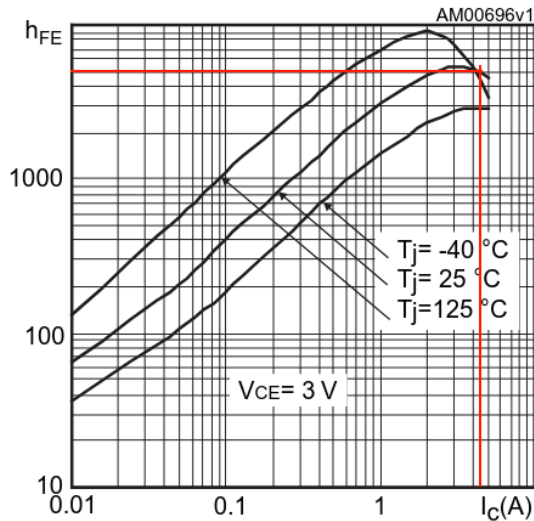


Figure 7. Base-emitter saturation voltage (NPN)

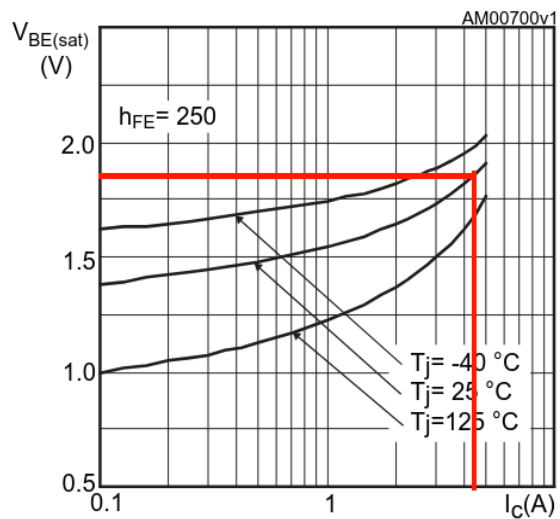


Abbildung 33: Kennlinien (Quelle: Datenblatt TIP120)

Als erstes würde ein TIP120 NPN in Erwägung gezogen, allerdings hat sich später herausgestellt das er bei einem Strom von 3,5 A gekühlt werden muss.

$$I_1 = 3,5 \text{ A}$$

$$h_{FE} = 1400$$

$$I_B = \frac{3.5}{1400} = 10 \text{ mA}$$

$$\alpha = \frac{h_{FE}}{2} = \frac{1400}{2} = 700$$

$$I_\alpha = \frac{I_1}{\alpha} = \frac{3.5}{700} = 5 \text{ mA}$$

$$U_{BE} = 1,85 \text{ V}$$

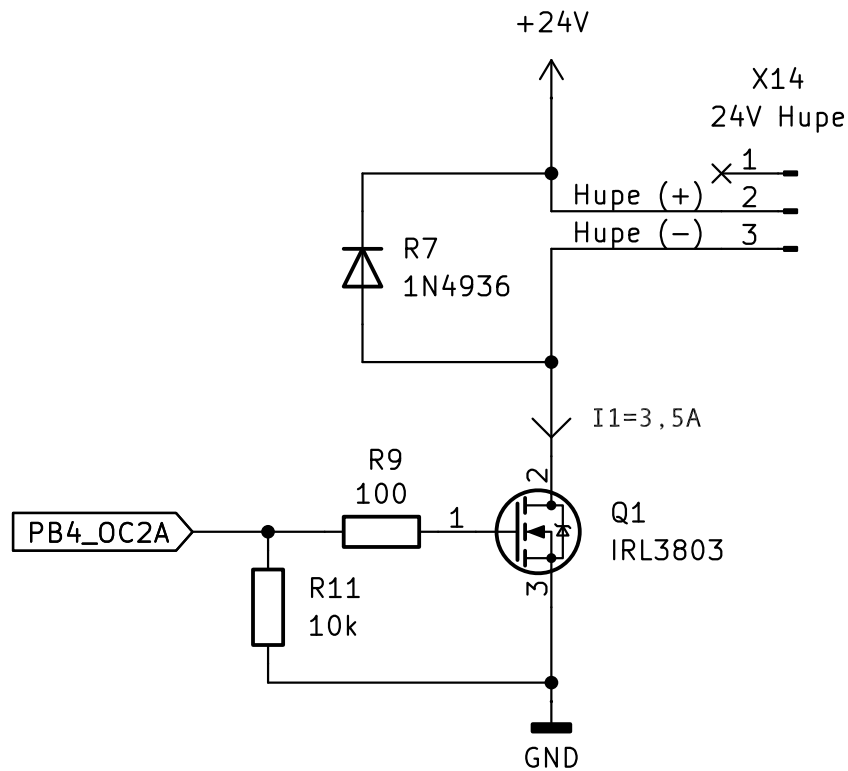
$$U_{MC} \in [4,2 \text{ V}; 5,1 \text{ V}]$$

$$R_x = \frac{U_{MCmin} - U_{BE}}{I_\alpha} = \frac{4,2 \text{ V} - 1,85 \text{ V}}{5 \text{ mA}} = 470 \Omega \pm 10 \%$$

$$I_{Bmax} = \frac{U_{MCmax} - U_{BE}}{R_{xmin}} = \frac{5,1 \text{ V} - 1,85 \text{ V}}{470 \Omega \cdot 0.9} = 7,68 \text{ mA}$$

$$I_{Bmin} = \frac{U_{MCmin} - U_{BE}}{R_{xmin}} = \frac{4,2 \text{ V} - 1,85 \text{ V}}{470 \Omega \cdot 1.1} = 4,55 \text{ mA}$$

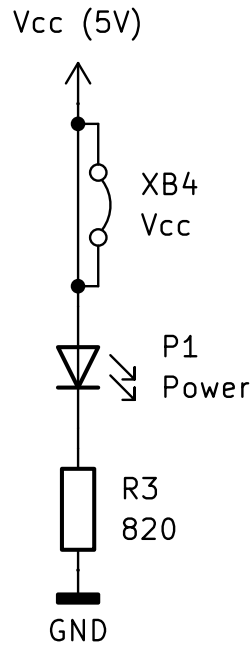
3.9.2 Finaler Transistor (IRL3803)



Bei dem IRL3803 handelt es sich um einen Logik-Level-MOSFET, das heißt das er bei 5V fast vollständig durchschaltet. Für die Widerstände werden typische Werte genommen, wobei für R9 ein möglichst geringer Widerstand verwendet wird um ein PWM Signal zu ermöglichen. R10 dient als Pull-Down-Widerstand. R9 als Strom begrenzer, da ein MOSFET sich wie ein Kondensator am Gate verhält und zu Beginn eines Schaltzykluses viel Strom zieht.

Die Diode dient als Freilaufdiode, falls eine induktive Last angeschlossen wird.

3.10 Power-LED Dimensionierung



Schaltplanausschnitt 2:
Power-LED

Die LED P1 zeigt an ob die Vcc Spannung mit 5V anliegt. Es handelt sich hier um eine grüne Low-Power-LED mit sehr geringen Stromverbrauch.

$$\begin{aligned}U &= 5 \text{ V} \\U_{P1} &= 1,7 \text{ V} \\I_{P1} &= 4 \text{ mA} \\U_{R1} &= U - U_{P1} = 5 \text{ V} - 1,7 \text{ V} = 3,3 \text{ V} \\R_1 &= \frac{U_{R1}}{I_{P1}} = \frac{3,3 \text{ V}}{4 \text{ mA}} \approx 825 \Omega \Rightarrow R_1 = 820 \Omega \pm 10 \% \text{ E12-Reihe}\end{aligned}$$

Bei einer Status-LED sind 10% Schwankung des Stromes zu vernachlässigen.

3.11 Lenkrad

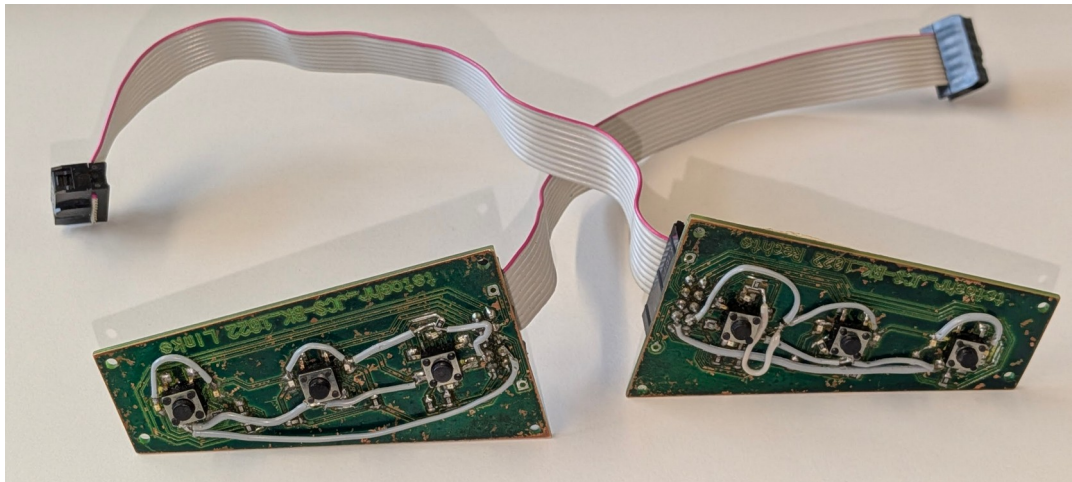


Abbildung 34: Lenkrad Taster

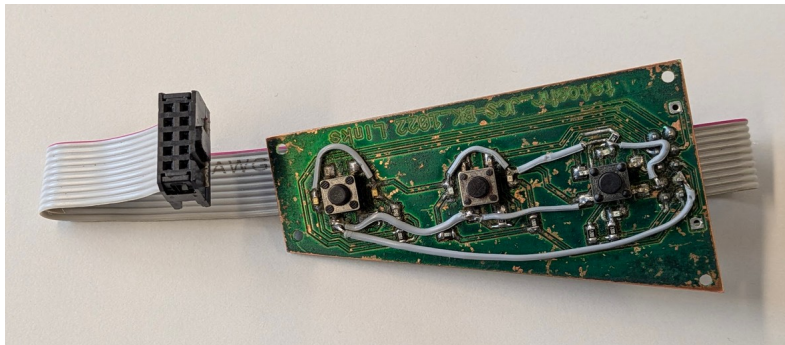


Abbildung 36: Taster-Anschluss



Abbildung 35: Start/Stop/Rückwärts

Die Taster werden im Lenkrad eingebaut. Der Start/Stop/Rückwärts Schalter legt den Vorwärts- bzw. Rückwärtsgang fest und sendet diese Information an den Antrieb weiter.

3.12 Cockpit-Platine

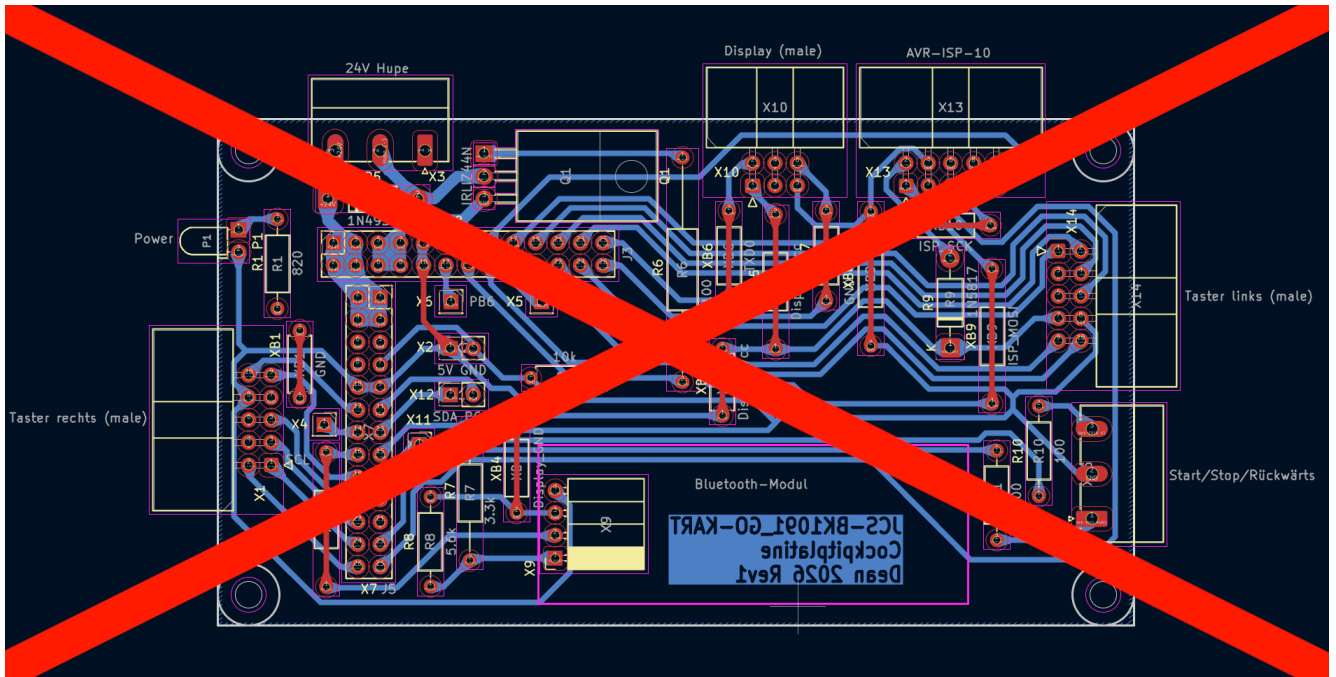


Abbildung 37: Entwurf der Cockpitplatine (Rev1)

Als Erstes wurden alle benötigten Anschlüsse eingeplant, dann wurde daraus ein Schaltplan erstellt. Bei der Layout-Erstellung wurden die Anschlüsse als Erstes so positioniert wie sie real Sinn machen. Anschließend wurden die Leitungen verbunden.

Nach der ersten Version (Rev1) fiel auf das die Anschlüsse für die Taster besser oben anstatt von der Seite positioniert werden. Also wurde das Layout erneut erstellt und die aktuelle und finale Version (Rev2) ist entstanden.

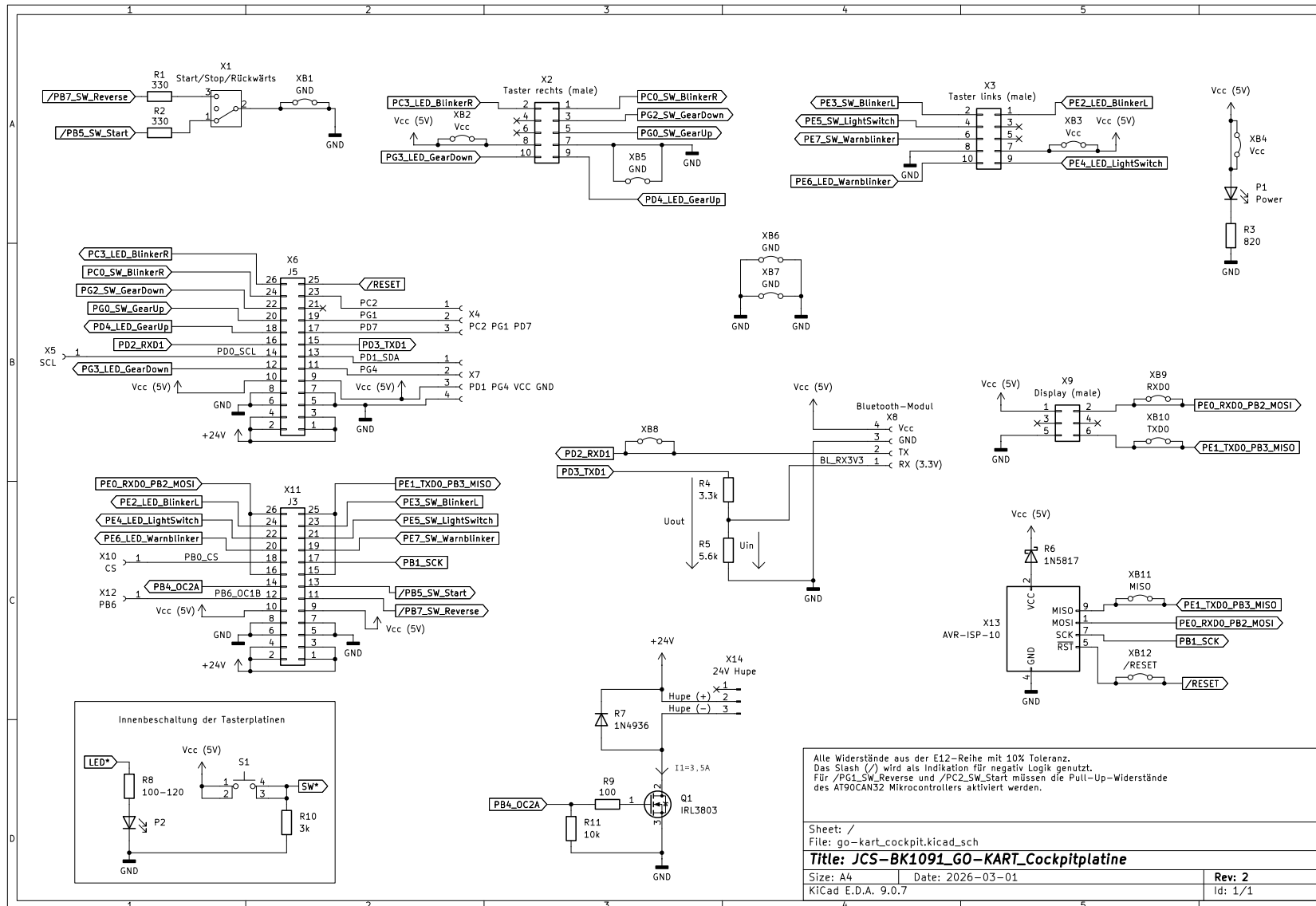


Abbildung 38: Schaltplan der Cockpitplatine Rev 2

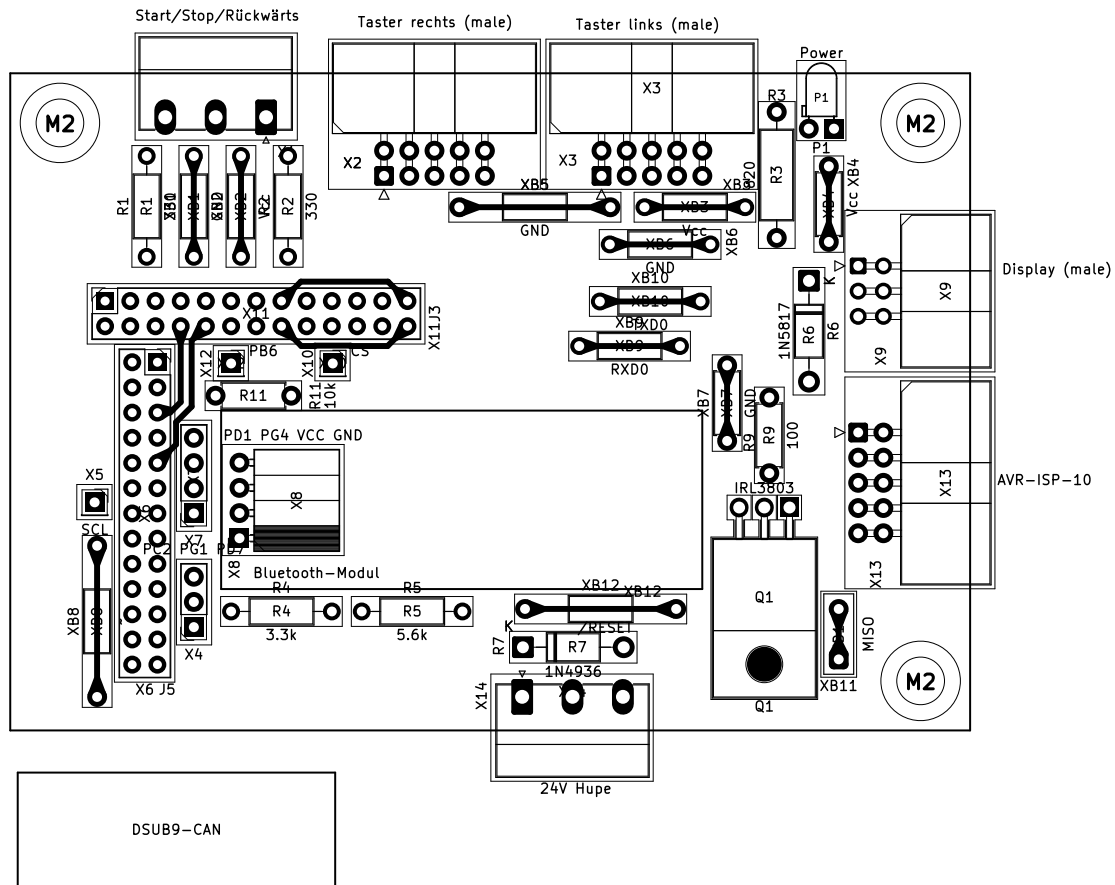


Abbildung 40: Bauteilansicht (top) der Cockpitplatine Rev 2

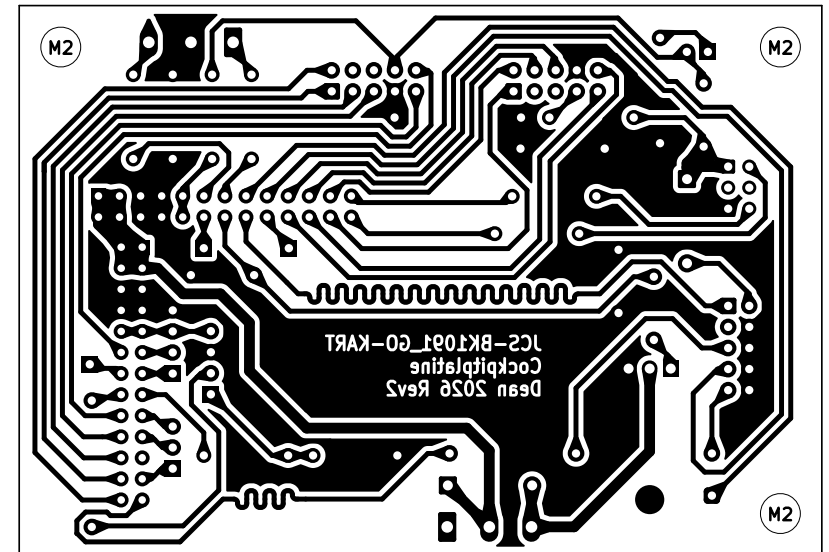


Abbildung 39: Leiterbahnen (bottom) der Cockpitplatine Rev 2

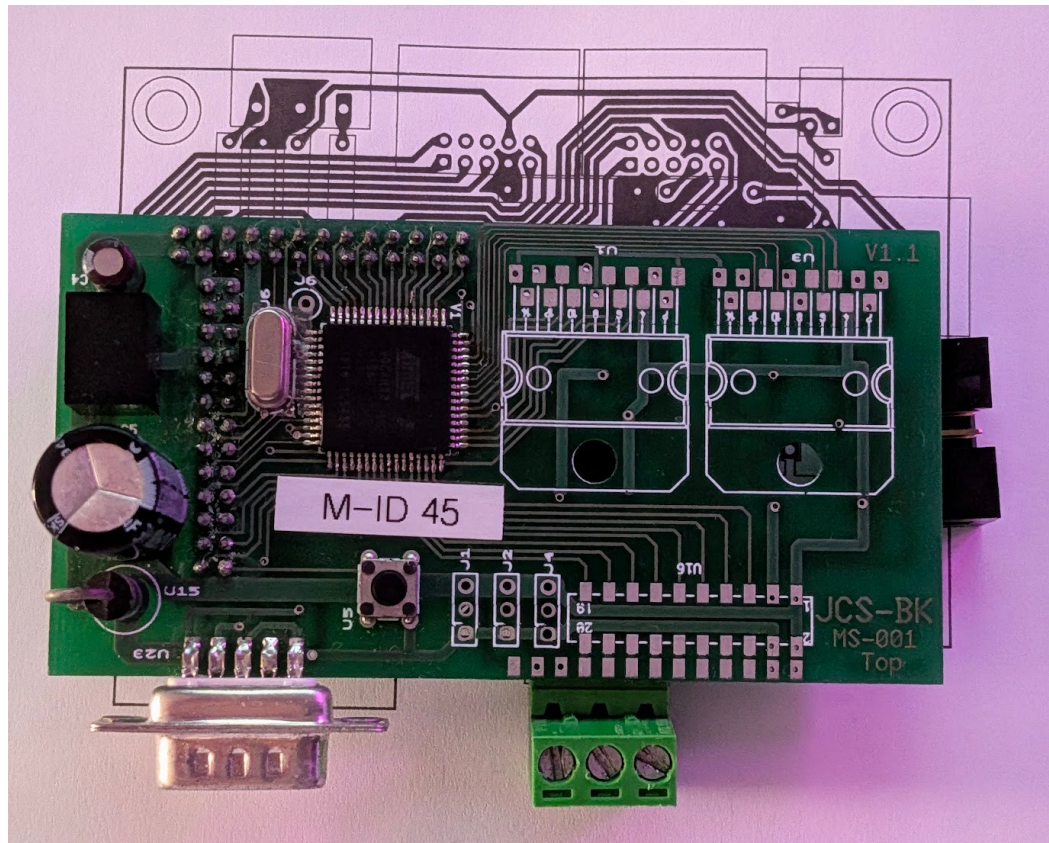


Abbildung 41: Abmessung Ausdruck

Um die Abmessungen besser einschätzen zu können, wurde das Layout auf Papier in original Größe ausgedruckt.

3.12.1 Prüfprotokoll

Bedingung	geprüft von		
	Dean	Henry	Tammo
1. Schaltplan			
Title ist sinnvoll.	x	x	x
Beschreibung ist verständlich.	x	x	x
Alle Bauteile sind korrekt bezeichnet.	x	x	x
Dimensionierung des Vorwiderstands der Power LED ist nachvollziehbar.	x	x	x
Dimensionierung des Widerstands für den Darlingtontransistor der Hupe ist nachvollziehbar.	x	x	x
Hupen Schaltung korrekt (MC Pin, Transistor und Diode).	x	x	x
Dimensionierung des Spannungsteiler für das Bluetooth-Modul ist nachvollziehbar.	x	x	x
Alle verwendeten Anschlüsse der AT90CAN32 Platine mit dem Schaltplan überprüft.	x	x	x
Alle Messbrücken sind passend beschriftet.	x	x	x
Alle verwendeten Anschlüsse der AT90CAN32 Platine nachgemessen	x		
Anschlussstecker des Displays durch Messen überprüft.	x		
Anschlussstecker der linken Tasterplatine durch Messen überprüft.	x		
Anschlussstecker der rechten Tasterplatine durch Messen überprüft.	x		
2. Platine			
Jedes Bauteil ist montierbar.	x	x	x
Leitungen welche 4 Ampere Leiten haben 1,3 mm Breite und einen Abstand von 0,6 mm.	x	x	x
Restliche Leitungen haben eine Breite von 0,6 mm Breite und einen Abstand von 0,5 mm.	x	x	x
DRC (Design Rules Checker) wurde durchgeführt.	x	x	x
Platinen beschriftung wurde passend gewählt und ist gespiegelt.	x	x	x
Position der Ausgänge und der LED wurden passend gewählt.	x	x	x
Alle Footprints wurden nachgemessen.	x		
Jedes Bauteil ist vorhanden.	x		

Tabelle 7: Prüfprotokoll von der Cockpit-Platine

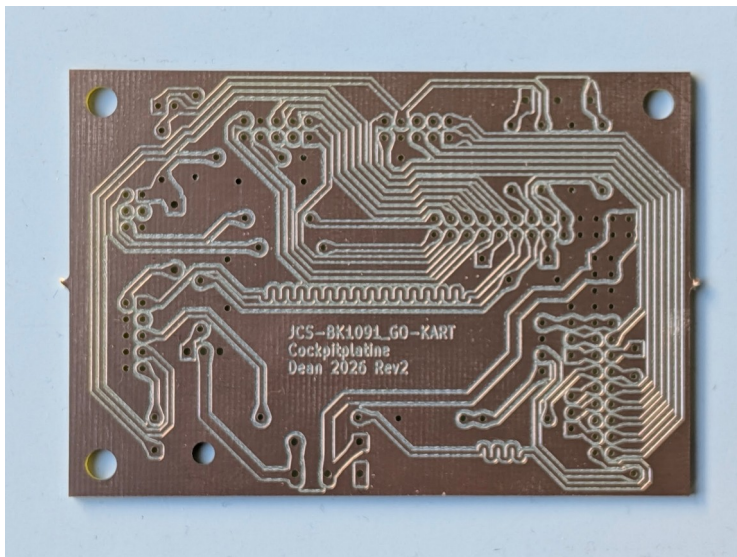


Abbildung 43: gefräste Kupferplatte

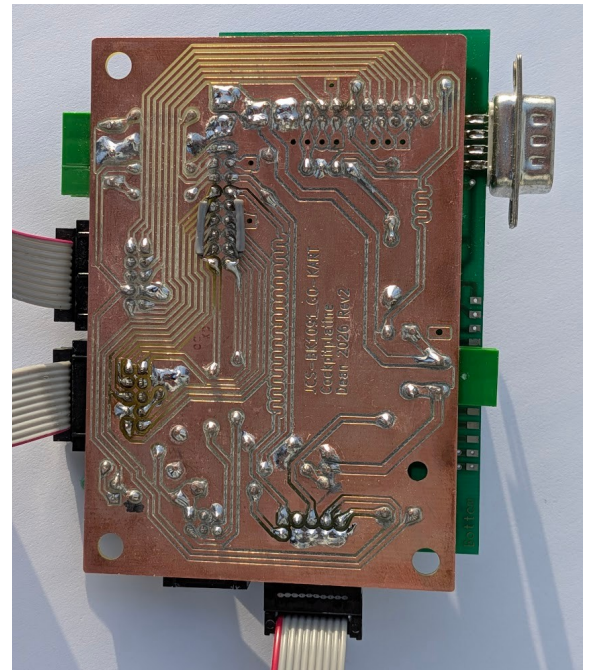


Abbildung 42: Bestückte Platine

Nachdem die sehr häufig geprüfte Platine gedruckt wurde, fiel auf das der Mikrocontroller sich nicht programmieren lies. Relativ schnell war der Fehler klar, der MOSI bzw. MISO Anschluss des SPI ist nicht für ISP (In-System-Programming), stattdessen muss TXD0 und TXD1 verwendet werden. Dieser Fehler würde durch eine Brücke behoben. Alle gezeigten Schaltpläne in dieser Dokumentation haben die Brücke bereits eingezeichnet.

3.13 Cockpit-Programm

Beim Programmieren wurde darauf geachtet das nach EVA-Prinzip gearbeitet wird. Das heißt das erst alles Eingelesen, verarbeitet und anschließend Ausgegeben wird. Außerdem wurden Variablennamen extra ausführlich gewählt und ersetzen so häufig Kommentare, wie es auch schon in den 1.3 Code Konventionen (S. 4) festgelegt wurde. Hier ist Beispielhaft die Hauptschleife des Cockpits, der gesamte Code ist ebenfalls verfügbar.

```
int main(void) {
    init();
    sei();

    // test_display();

    GearState prev_gear_state = gear_state;
    LightState prev_light_state = light_state;
    bool prev_blinker_left = false;
    bool prev_horn = false;
    bool prev_blinker_right = false;

    while (true) {
        // --- Einlesen ---
        const bool blinker_left = blinker_left_live;
        const bool blinker_right = blinker_right_live;
        horn = GET(SW_WARNBLINKER_HORN);

        if (GET(SW_MOTOR_START_LA)) {
            if (GET(SW_MOTOR_REVERSE_LA)) {
                motor_state = STOP;
            } else {
                motor_state = REVERSE;
            }
        } else {
            if (GET(SW_MOTOR_REVERSE_LA)) {
                motor_state = FORWARD;
            } else {
                motor_state = STOP;
                uart_puts("error: Vorwärts- und Rückwärtsfahren nicht
gleichzeitig "
                        "möglich. Taster auf defekt überprüfen.\n");
            }
        }

        // --- Verarbeiten ---
        if (light_switch) {
            light_switch = false;
            light_state = (light_state + 1) % LIGHT_STATE_COUNT;
        }

        if (gear_up) {
            gear_up = false;

            if (gear_state < GEAR_STATE_COUNT - 1) {
                gear_state++;
            }
        }
    }
}
```



```

    }
} else if (gear_down) {
    gear_down = false;

    if (gear_state > 0) {
        gear_state--;
    }
}

// --- Ausgeben ---
if (blinker_left != prev_blinker_left) {
    prev_blinker_left = blinker_left;
    cd_set_blinker_left(blinker_left);

    if (blinker_left) {
        SET(LED_BLINKER_LEFT);
    } else {
        RESET(LED_BLINKER_LEFT);
    }
}

if (light_state != prev_light_state) {
    cd_set_light(light_state);
}

if (horn != prev_horn) {
    if (horn) {
        SET(LED_WARNBLINKER_HORN);
        SET(HORN_PWM);
    } else {
        RESET(LED_WARNBLINKER_HORN);
        RESET(HORN_PWM);
    }
}

if (gear_state != prev_gear_state) {
    cd_set_gear(gear_state);
}

if (blinker_right != prev_blinker_right) {
    cd_set_blinker_right(blinker_right);

    if (blinker_right) {
        SET(LED_BLINKER_RIGHT);
    } else {
        RESET(LED_BLINKER_RIGHT);
    }
}

// some button LEDs are set in the ISR for simplicity

if (light_state != prev_light_state ||
    blinker_right != prev_blinker_right) {
    can_t msg_blinker = {
        .flags.rtr = false,

```

```

        .flags.extended = true,
        .length = 1,
    };

    uint8_t blinker_front_command = blinker_right << BLINKER_BIT;

    switch (light_state) {
    case LIGHT_OFF:
        break;
    case LIGHT_DAY:
        blinker_front_command |= 1 << LOWER_LIGHT_BIT;
        break;
    case LIGHT_LOW_BEAM:
        blinker_front_command |=
            1 << UPPER_WEAK_LIGHT_BIT | 1 << LOWER_LIGHT_BIT;
        break;
    case LIGHT_HIGH_BEAM:
        blinker_front_command |=
            1 << UPPER_FULL_LIGHT_BIT | 1 << LOWER_LIGHT_BIT;
        break;
    case LIGHT_STATE_COUNT:
        break; // unreachable
    }

    uart1_puts("info: sende Befehl zu Blinker vorne rechts:");
    uart1_puthex(blinker_front_command);
    uart1_putc('\n');

    msg_blinker.id = CAN_ID_TX_BLINKER_FRONT_RIGHT;
    msg_blinker.data[0] = blinker_front_command;
    can_send_message(&msg_blinker);

    uint8_t blinker_back_command = blinker_right << BLINKER_BIT;

    uart1_puts("info: sende Befehl zu Blinker hinten rechts\n");
    msg_blinker.id = CAN_ID_TX_BLINKER_BACK_RIGHT;
    msg_blinker.data[0] = blinker_back_command;
    can_send_message(&msg_blinker);
}

if (light_state != prev_light_state || blinker_left !=
prev_blinker_left) {
    can_t msg_blinker = {
        .flags.rtr = false,
        .flags.extended = true,
        .length = 1,
    };

    uint8_t blinker_front_command = blinker_left << BLINKER_BIT;

    switch (light_state) {
    case LIGHT_OFF:
        break;
    case LIGHT_DAY:
        blinker_front_command |= 1 << LOWER_LIGHT_BIT;

```

```

        break;
    case LIGHT_LOW_BEAM:
        blinker_front_command |=
            1 << UPPER_WEAK_LIGHT_BIT | 1 << LOWER_LIGHT_BIT;
        break;
    case LIGHT_HIGH_BEAM:
        blinker_front_command |=
            1 << UPPER_FULL_LIGHT_BIT | 1 << LOWER_LIGHT_BIT;
        break;
    case LIGHT_STATE_COUNT:
        break; // unreachable
    }

    uart1_puts("info: sende Befehl zu Blinker vorne rechts:");
    uart1_puthex(blinker_front_command);
    uart_putc('\n');

    msg_blinker.id = CAN_ID_TX_BLINKER_FRONT_LEFT;
    msg_blinker.data[0] = blinker_front_command,
    can_send_message(&msg_blinker);

    uint8_t blinker_back_command = blinker_left << BLINKER_BIT;

    uart1_puts("info: sende Befehl zu Blinker hinten rechts\n");
    msg_blinker.id = CAN_ID_TX_BLINKER_BACK_LEFT;
    msg_blinker.data[0] = blinker_back_command,
    can_send_message(&msg_blinker);
}

prev_gear_state = gear_state;
prev_light_state = light_state;
prev_horn = horn;
prev_blinker_right = blinker_right;
}

emergency_loop:
    goto emergency_loop;
}

```

4 Grundlagen

In diesem Kapitel werden die Grundlagen zur bedienung der Software erklärt. So das man dieses Projekt weiter entwickeln oder warten kann. Dabei richtet sich diese Anleitung an Linux-Nutzer, einige Programme sind ebenfalls für andere System wie Windows verfügbar.

4.1 Kompilieren und Hochladen

Benötigtes Paket:

- cmake

Als erstes müssen die USB-Rechte eingestellt werden.

```
$ sudo dmesg
```

```
usb 1-2: New USB device found, idVendor=16c0, idProduct=05dc
```

```
usb 1-2: Product: USBasp
```

```
https://wiki.archlinux.org/title/
```

```
Udev#Allowing_regular_users_to_use_devices
```

```
$ sudo -e /etc/udev/rules.d/71-usbasp.rules
```

```
SUBSYSTEMS=="usb", ATTRS{idVendor}=="16c0", ATTRS{idProduct}  
=="05dc", MODE="0660", TAG+="uaccess"
```

CMake und Make sind Werkzeuge, die den Bau von Programmen steuern, ähnlich wie eine IDE (Microchip Studio), nur über die Kommandozeile. CMake erzeugt Baupläne, die das System und den Compiler kennen, und Make führt diese Pläne aus, übersetzt den Code und erstellt die ausführbaren Dateien.

```
$ cmake -B build
```

Dieser Befehl erzeugt in einem neuen Verzeichnis namens „build“ alle notwendigen Dateien, die für den Bau des Projekts benötigt werden. CMake liest die „CMakeLists.txt“ Datei aus dem „src“ Verzeichnis, plant welche Dateien kompiliert werden müssen, und erstellt die Bauanweisungen (Makefile).

```
$ make
```

Make liest die von CMake erzeugten Makefiles und führt die Anweisungen aus. Es kompiliert die Quellcodes in Maschinencode und erstellt daraus die fertigen Programme oder Bibliotheken. Make sorgt dafür, dass nur die geänderten Dateien neu gebaut werden, was Zeit spart.

4.2 UART Übertragung

4.2.1 Text übertragen

Benötigtes Paket:

- minicom

1. USB-zu-Seriell-Konverter Geräte Schnittstelle finden (z. B. /dev/ttyUSB0 oder /dev/ttyACM0):

```
$ ls /dev/tty*
```

2. Serielle Übertragung öffnen. In diesem Fall mit dem Tool „minicom“ und eine BAUD-Rate von 9600 bps:

```
$ minicom -D /dev/ttyUSB0 -b 9600
```

4.2.2 Rohdaten übertragen

Um Binär oder Hexadezimal über UART zu senden kann man nicht *minicom* verwenden. In diesem Beispiel wird das Standard-Linux-Interface genutzt.

1. BAUD-Rate einstellen (z. B. 9600 bps):

```
$ stty -F /dev/ttyUSB0 9600
```

2. Nachricht senden (z. B. für das Cockpit-Display):

```
$ printf 'bL.pic=1\xFF\xFF\xFF' > /dev/ttyUSB0
```

Hierbei ist die Nachricht: „bL.pic=1“ und drei Mal FFh zum beenden des Befehls.

4.2.3 Bluetooth Übertragung

In diesem Kapitel wird erklärt wie man eine Verbindung mit dem Bluetooth-Modul (siehe 3.4 Bluetooth-Modul, S. 22) aufnehmen kann.

Warnung: IOS unterstützt keine Verbindung zu SPP (Serial Port Profile), sondern nur BLE (Bluetooth Low Energy). Da es sich hier um SPP handelt, ist keine Verbindung mit dem iPhone oder iPad möglich.

Benötigte Pakete:

- minicom
- bluez
- bluez-deprecated-tools

Die Arch Wik¹ hat eine ausführliche Erklärung wie man sich mit einem Bluetooth-Gerät verbindet.

Nachdem eine Bluetooth-Verbindung hergestellt wurde, muss sie einer device-Datei gebunden werden:

```
$ sudo rfcomm bind rfcomm0 <MAC Adresse des Bluetooth-Moduls>
```

Anschließend kann eine serielle Übertragung (siehe 4.2 UART Übertragung, S. 55) mit einer BAUD-Rate von 9600 bps hergestellt werden:

```
$ minicom -D /dev/rfcomm0 -b 9600
```

Das Blinken auf dem Bluetooth-Modul sollte aufhören und die Status-LED sollte konstant Leuchten.

¹ mehr Informationen zu Bluetooth mit Linux in der Arch Wiki [AW01]

4.3 CAN Sniffing (PCAN-USB)

Benötigte Pakete:

- can-utils

Als CAN-Interface für USB wird der PCAN-USB IPEH-002021 von PEAK-System verwendet.

Der D-Sub stecker von allen Modulen ist nach CiA® 106 (siehe Abbildung 9: männlicher CAN D-Sub Anschluss nach CiA® 106 (Quelle: PCAN-USB Anleitung), S. 16)



Abbildung 44: PCAN-USB (Quelle: peak-system.com)

4.3.1 Empfangen

Der folgende Befehl aktiviert das Netzwerk-Interface `can0` als CAN-Bus-Schnittstelle und setzt dabei die Übertragungsrate auf 125 kbit/s. Dieser Schritt muss nur einmal durchgeführt werden.

```
$ sudo ip link set can0 up type can bitrate 125000
```

Anschließend können Nachrichten des CAN-Busses ausgegeben werden.

```
$ candump can0
```

Mögliche Ausgabe:

```
can0 400 [8] 01 02 02 03 04 05 06 07
```

4.3.2 Senden

Beispiel mit ID = 4AAh und erstes Datenbyte = 0Fh, zweites Datenbyte = 55h:

```
$ cansend 4AA#0F55
```

4.3.3 Fehlerbehebung

Eventuell muss der Treiber von PEAK-System manuell installiert werden.

Ob ein Treiber installiert ist kann man zum Beispiel über die System-Logs herausfinden. Da der Treiber je nach Distribution auch anders als `peak_usb` benannt sein kann, muss man eventuell den gesuchten Eintrag mit `can0`, `pcan` oder `peak` suchen.

```
$ sudo dmesg | grep -i peak_usb
```

Wenn die Ausgabe so ähnlich aussieht ist der Treiber installiert:

```
peak_usb 3-2:1.0: PEAK-System PCAN-USB adapter hwrev 28 serial  
peak_usb 3-2:1.0 can0: attached to PCAN-USB channel 0  
usbcore: registered new interface driver peak_usb
```

Falls der Treiber nicht installiert ist kann man ihn über den Package-Manager des Systems herunterladen oder über die Webseite von PEAK-Systems.

5 Fazit

5.1 Kosten

	Preis pro Stunde	Arbeitszeit (h)	Preis
Dokumentation	15,00 €	76	1.140,00 €
Entwicklungszeit	25,00 €	154	3.850,00 €
	gesamt	230	<u>4.990,00 €</u>

Tabelle 8: Arbeitszeit Kosten

	Anzahl	Preis pro Stück	Preis
5x2 gewinkelte Buchse	3	0,90 €	2,70 €
3x2 gewinkelte Buchse	1	0,80 €	2,40 €
13x2 Buchse	2	1,00 €	2,00 €
Widerstände der E12-Reihe	7	0,10 €	0,70 €
Kupferplatte	1	6,00 €	6,00 €
Nextion Display	1	95,00 €	95,00 €
2-pol Anschluss	1	1,10 €	1,10 €
3-pol Anschluss	1	1,20 €	1,20 €
LED	1	0,10 €	0,10 €
		gesamt	<u>111,20 €</u>

Tabelle 9: Material Kosten

Die gesamten Kosten belaufen sich auf 5101,20 €.

5.2 Reflexion

Der Terminplan war von Anfang an sehr knapp bemessen und die zusätzlichen Hausaufgaben, welche uns an das Projekt führen sollten, haben mich zu mindestens sehr gebremst, da man jedes mal den Prozess dokumentieren musste. Ich hätte es besser gefunden hätten wir frei am Projekt arbeiten können. Die geplanten blauen Felder (siehe 7.2 Terminplan, S. 61) passen ziemlich gut den tatsächlichen Arbeitszeiten überein. Gerade mit dem Blick auf den Ergebnissen den vergangenen Jahren bin ich zufrieden mit meinem Beitrag zum Go-Kart. Alle Aufgaben aus dem Lasten- und Pflichtenheft wurden realisiert.

Wenn ich heute nochmal das Projekt beginnen würde, würde ich weniger Zeit mit den alten Dokumentation verbringen, diese haben mir oft nicht weiter geholfen und da man den Angaben nicht vertrauen konnte, musste hinterher fasst alles Ausgemessen werden.

Insbesondere die Teamarbeit hat meine Kompetenzen erweitert. Das Team war sehr engagiert und man konnte gut zusammen arbeiten.

5.3 Ausbau

Diese Themen sollten von dem Nachfolger bearbeitet werden:

1. Gehäuse für Cockpit mit 3D-Drucker erstellen.
2. Tasterplatinen für Lenkrad erneuern und die Kabelbrücken somit ersetzen.
3. Gehäuse für die Taster im Lenkrad erstellen.
4. Steuerprogramm für den PC um visuell auszuwerten und Befehle zu senden.
5. Restliche CAN-Nachrichten in das Cockpit-Programm hinzufügen

6 Eidesstattliche Erklärung

Hiermit versichere ich, dass ich die vorliegende Dokumentation selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht.

Ferner versichere ich, dass die vorliegende Arbeit noch nicht in einem anderen Prüfungsverfahren vorgelegen hat.

X

Dean Schneider (Auftragnehmer) & Datum



7 Anhang

7.1 Projekte der Vorgänger

Hiervon sind hauptsächlich JCS-BK_1025 (Schaltpläne für Lichtanlage), JCS-BK_1049 (Lichtanlage) und JCS-BK_1046 (Lenkradknöpfe, Cockpitdisplay) von Bedeutung.

Jahr	Projektbezeichnung	Projektfunktion	Auftragnehmer
2023	JCS-BK_1071_GO-KART Blinker_Lenkrad_Anzeige	Blinkerinbetriebnahme, Lenkradanbindung und die Visualisierung per PC/Laptop, mit CAN-BUS Anbindung, Basis ist JCS-BK_MS-063, JCS-BK_MS-123	Elias Hahne
2021	JCS-BK_1046_GO-KART Fahrzeugarmaturenbrett	MC basiertes Armaturenbrett ist zu fertigen und die Visualisierung per Tablet / Handy, Basis ist JCS-BK_MS-063, JCS-BK_MS-123	Christian Barth
2021	JCS-BK_1049_GO-KART Lichtanlage	Es sind Front- und Rückleuchten zu erstellen und die Blinker sollen Als Lauflicht ausgeführt werden, Basis ist JCS-BK_MS-060, JCS-BK_MS-130	Robin Kraus
2020	JCS-BK_1034_GO-KART Fahrzeugarmaturenbrett	MC basiertes Armaturenbrett ist zu fertigen und die Visualisierung per Tablet / Handy, Basis ist JCS-BK_MS-063, JCS-BK_MS-123	Daniel Plempe
2020	JCS-BK_1037_GO-KART Lichtanlage	Es sind Front- und Rückleuchten zu erstellen und die Blinker sollen Als Lauflicht ausgeführt werden, Basis ist JCS-BK_MS-060, JCS-BK_MS-130	Daniel Briem
2019	JCS-BK_1022_GO-KART Fahrzeugarmaturenbrett	MC basiertes Armaturenbrett ist zu fertigen und die Visualisierung per Tablet / Handy, Basis ist JCS-BK_MS-063, JCS-BK_MS-123	Tim Stöhr
2019	JCS-BK_1025_GO-KART Lichtanlage	Es sind Front- und Rückleuchten zu erstellen und die Blinker sollen Als Lauflicht ausgeführt werden, Basis ist JCS-BK_MS-060, JCS-BK_MS-130	Christoph Waßmuth
2018	JCS-BK_1005_GO-KART Fahrzeugarmaturenbrett	MC basiertes Armaturenbrett ist zu fertigen und die Visualisierung per Tablet / Handy, Basis ist JCS-BK_MS-063, JCS-BK_MS-123	Lukas Guse
2018	JCS-BK_1008_GO-KART Lichtanlage	Es sind Front- und Rückleuchten zu erstellen und die Blinker sollen Als Lauflicht ausgeführt werden, Basis ist JCS-BK_MS-060, JCS-BK_MS-130	Luis Nolte
2017	JCS-BK_MS-123_GO-KART Fahrzeugarmaturenbrett	MC basiertes Armaturenbrett ist zu fertigen und die Visualisierung per Tablet / Handy, Basis ist JCS-BK_MS-063	Nils Kölle
2017	JCS-BK_MS-130_GO-KART Lichtanlage	Es sind Front- und Rückleuchten zu erstellen und die Blinker sollen Als Lauflicht ausgeführt werden, Basis ist JCS-BK_MS-060	Johannes Wachs
	JCS-BK_MS-060_GO-KART Lichtanlage	Front- und Rückleuchten sind zu fertigen	Philipp Junge, Jan Schmidt
	JCS-BK_MS-063_GO-KART Fahrzeugarmaturenbrett	JAVA-Tablet und MC Armaturenbrett ist zu fertigen	Patrick Schwarz

Tabelle 10: alte Projekte

7.2 Terminplan

[illegible]

Tabelle 11: Terminplan 01.09. - 19.09.2025

[illegible]

7.3 Meilensteine

Start-datum	Ziel-datum	Zeit (h)	Thema (Aufgabe)
01.09.25	11.09.25		Festlegen des Themas und der Projektteams (Informationssammlung, Pflichtenheft zur Aufgabenfixierung, Aufgabenverteilung im Team und Anfertigung des Detailterminplans)
12.09.25	08.10.25	18	<u>Abgabe des Pflichtenhefts</u>
08.10.25	30.11.25	34	Schaltplanerstellung / Programmierung von Modulen / Testaufbau / Modultests
08.10.25	30.11.25	11	<u>Abgabe der Draft 1 - Dokumentationsentwurf</u>
31.11.25	23.01.26	5	Statusermittlung / Terminplanprüfung
31.11.25	23.01.26	7	Schaltplan- und Layouterstellung / Programmierung von Modulen Testaufbau / Modultests / Dokumentation
31.11.25	23.01.26	15	Programmtests, Dokumentation, Fehleranalyse, Fehlerbeseitigung
31.11.25	23.01.26	20	Abgabe der Draft2-Dokumentation 7:35 keine Verlängerung
23.01.26	13.03.26	85	Programmtests, Dokumentation, Fehleranalyse, Fehlerbeseitigung
08.02.26	25.02.26	2	Präsentation der Ergebnisse als Impress-Vortrag
12.02.26	13.02.26	15	<u>Präsentation der Zwischenstände am Tag der offenen Tür</u>
10.03.26	13.03.26	10	Abgabe des Projekts mit Abschluss-Dokumentation 24:00
16.03.26	20.03.26		Leistungsüberprüfung / Produktabnahme

Tabelle 15: Meilensteine

7.4 Tagesziele

Nr.	Datum	Zeit (h)	Tagesziel	Reflexion
1	2025-09-12	2,0	Pflichtenheft Layout erstellen	Deckblatt und ungefähres Design fertig. Titelbild muss eingefügt werden.
2	2025-09-22	4,5	Nach Erkältung an Pflichtenheft weiterarbeiten.	- Lastenheft eingefügt - Projekte der Vorgänger herausgesucht - Bereits vorhandene Elemente und meine Aufgaben herausgeschrieben
3	2025-09-25	1.5	Pflichtenheft fortführen, insbesondere alle vorgegeben Tabellen einfügen (Meilensteine, Terminplan, ...)	Terminplan eingefügt
4	2025-09-26	3,0	Pflichtenheft weiter ausformulieren.	50 Prozent
5	2025-09-27	4,0	Pflichtenheft Aufgabenstellung definieren. Eventuell Zeitplan anpassen.	Aufgabenstellung weiter definiert und die letzten drei Vorgänger-Projekte analysiert.
6	2025-09-28	6,0	Pflichtenheft erste Version	Erste Version fertiggestellt.
7	2025-10-08	3,0	Pflichtenheft korrigieren.	Erfolgreich
8	2025-11-06	1,5	Kegelbahn-Anzeige ansteuern	Schaltplan analysiert, Anschlüsse festgestellt
9	2025-11-07	1,5	- Kegelbahn-Anzeige ansteuern - OLED Display ansteuern	Erfolgreich angesteuert
10	2025-11-08	3,0	Bluetooth Modul ansteuern	Kann mit USB angesteuert werden.
11	2025-11-09	6,0	Bluetooth Modul ansteuern	Kann vollständig angesteuert werden und vollständig dokumentiert.
12	2025-11-10	6,0	Display testen und ansteuern	Das Display wurde mit 14V gebraten. Ich habe ein Ersatzdisplay bekommen. Allerdings habe ich heute größtenteils den anderen geholfen und bin daher nicht weiter gekommen.
13	2025-11-11	6,0	Hausaufgaben zu den Kegelbahnanzeigen bearbeiten	Punkteanzeige fertig
14	2025-11-12	6,0	Wurfanzeige für die Kegelbahn	Muss nur noch hochgeladen werden.
15	2025-11-13	6,0	- Hausaufgabe zur Kegelbahn fertigstellen - kaputtes Display messen - AT90CAN Platine analysieren	- Hausaufgabe fertig - AT90CAN Platine wurde analysiert, allerdings funktioniert Hochladen aufgrund von avrdude noch nicht. Ich muss noch Feststellen, welchen Typ der Programmierer hat. - kaputtes Display messe ich zu Hause
16	2025-11-27 bis 2025-11-29	6,0	Hausaufgabe TI_85_Arduino_CAN-BUS_Test nachholen. Testaufbau für CAN-Bus mit dem MCP2515 Modul.	Ich brauche ein zweites MCP2515 (und TJA1050) Modul um einen richtigen Test durchführen zu können. Ich kann also erst in der Schule weiterarbeiten.
17	03.12.25	4,0	OLED Display ansteuern	Funktioniert mit Bibliothek. Außerdem Datenblatt analysiert, um die Einstellungen zu verstehen und eventuell selbst die Befehle zu senden.
18	05.12.25	1,5	CAN Bus Test: Senden und Empfangen	CAN Bus Analyzer (PCAN) mit Software zum laufen bringen. Nicht abgeschlossen
19	05.12.25	3,0	CAN Bus Test: Senden und Empfangen	Senden und Empfangen über ein CAN USB Interface funktioniert
20	12.12.25	3,0	OLED Display ansteuern	An PWM und ADC gearbeitet
21	19.12.25	3,0	CAN-Bus senden	OLED-Display, PWM, Analog mit C fertig SPI funktioniert nicht mit ATmega328p
22	08.01.26	1,5	CAN-Bus-Nachricht senden	CAN-Bus-Nachricht senden und empfangen funktioniert.
23	09.01.26	5,0	CAN Nachrichten auf OLED anzeigen.	Erfolgreich. Die Nachrichten werden fortlaufend angezeigt.
24	15.01.26	1,5	Nextion Display ansteuern.	Das Nextion Display mit UART ansteuern und mit den anderen Bluetooth getestet.
25	16.01.26	3,0	Cockpit Display angesteuert	Programm zum Ansteuern geschrieben
26	19.01.26	3,0	Programm testen und Dokumentation schreiben über CAN-Bus, Display und Cockpit-Display.	Programm getestet

Tabelle 16: Tagesziele Teil 1

Nr.	Datum	Zeit (h)	Tagesziel	Reflexion
27	20.01.26	3,0	Dokumentation schreiben über CAN-Bus, Display und Cockpit-Display.	Schaltplan für Bluetooth, cmake Erklärung, CAN-Sniffing
28	22.01.26	3,0	Blinker Modul ansteuern	Blinker Modul muss neu programmiert werden, da das letzte Projekt (JCS-BK_1071) den Microcontroller mit einen Testprogramm programmiert hat. CAN-Bus-Steuerung
29	22.01.26	4,0	Dokumentation weiter schreiben	allgemeine Verbesserungen Cockpit-Display Kapitel erste Version OLED-Display Kapitel erste Version
30	23.01.26	7,0	Blinker Mikrocontroller testen Dokumentation Draft 2	Blinker LEDs getestet, als Nächstes muss das Programm zum Steuern über CAN-Bus erstellt werden. Draft 2 fertiggestellt
31	24.01.26	4,0	LEDs des Blinkers testen	Erfolgreich, allerdings scheint der MCU langsam nicht mehr programmierbar zu sein, es kommt häufig zu Fehlern beim hochladen.
32	27.01.26	3,0	AT90CAN mit USART1 und USART0 gleichzeitig	Abgebrochen
33	28.01.26	2,5	Cockpit Display testen mit Testprogramm und AT90CAN32.	Erfolgreich, es gab einen Bug in der Bibliothek für den AT90CAN32.
34	29.01.26	5,0	ATmega16M1 programmieren und schließlich CAN-Bus für die Blinker testen.	Dokumentation besprochen und USB Rechte eingestellt. Das Problem mit dem Programmierer und ATmega16M1 lag, daran das 3.3V anstelle von 5V eingestellt waren. Als Erstes wird der AT90CAN mit CAN-Bus getestet.
35	30.01.26	3,0	CAN-Bus mit AT90CAN32.	CAN-Bus-Test erfolgreich mit Dokumentation.
36	31.01.26	4,0	CAN-Bus mit ATmega16M1	Bug in der Bibliothek nicht gefunden allerdings eingekreist in der Interrupt routine (Er tritt nur auf wenn man Interrupts aktiviert).
37	01.02.26	7,0	CAN-Bus mit ATmega16M1 Blinker Steuerung	Nach weiteren 3 Stunden endlich den Bug in der Bibliothek gefunden. Mithilfe ISR(BADISR_vect) trat der Reset nicht mehr auf. Dadurch konnte ich den Reset auf die ISR(CANIT_vect) zurückführen. Nach genauer Recherche im Datenblatt erkannte ich das CANIT_vect nicht für den ATmega16M1 geeignet ist, sondern nur CAN_INT_vect. Durch einfaches Austauschen war der Bug behoben. Blinker Steuerung fertig nach den internationalen Normen und Dokumentation geschrieben.
38	04.02.26	2,0	UART für ATmega16M1 und steuerung des Blinkers über UART oder CAN-Bus.	Fertig, muss nur noch getestet werden.
39	09.02.26	6.5	Blinker UART testen mit Bluetooth Modul.	Erfolgreich Zusätzlich: Statusnachricht im Intervall senden, PWM für Rücklicht, beides muss getestet werden
40	10.02.26	4,0	neue Funktionen Testen Taster und Display planen, Display Bibliothek überarbeiten	Blinker Modul mit PWM morgen: CAN ID Tabelle, Taster und Display Platine
41	11.02.26	4,0	CAN ID Tabelle, Taster und Display Platine erstellen	CAN-ID Tabelle erste Version. Den anderen geholfen.
42	12.02.26	7,0	Tasterplatine Schaltung	Tasterplatine wird nicht erneuert. Verbindungsplatine Schaltung angefangen.
43	13.02.26	7,0	Tasterplatinen ausmessen	Schaltplan fertiggestellt
44	14.02.26	4,0	Testprogram für Cockpit erstellen	Erfolgreich
45	15.02.26	3,0	Platinenplan angefangen	etwa 50%
46	16.02.26	7,0	Platine fertigstellen	Erfolgreich. Prüfprotokoll wird in den nächsten Tagen von Henry und Tammo kontrolliert.
47	01.03.26	6,0	Anschlüsse auf der Platine besser positionieren.	Erfolgreich
48	02.03.26	4,0	Platine drucken und bestücken.	Gelötet und grundsätzlich getestet. Ein 5x2 wurde noch offen gelassen, bis klar ist ob noch ein besserer gewinkelter verfügbar ist.
49	04.03.26	2,0	Programmieranschluss funktioniert nicht.	Warum ist MOSI an RXD0 angeschlossen?

Tabelle 17: Tagesziele Teil 2

Nr.	Datum	Zeit (h)	Tagesziel	Reflexion
49	04.03.26	2,0	Programmieranschluss funktioniert nicht.	Warum ist MOSI an RXD0 angeschlossen?
50	05.03.26	5,0	Platine reparieren und testen.	MOSI und RXD0 bzw. MISO und TXD0 mit einer Brücke verbunden. Hupe mit Testprogram getestet. Pull-down-Widerstand eines Tasters war nicht eingebaut. Alle Taster und Bluetooth getestet, somit ist die gesamte Hardware getestet. Programm für Cockpit fertiggestellt. CAN-Bus-Kommunikation mit anderen Projekten ist noch offen.
51	07.03.26	5,0	Dokumentation überarbeiten	Abstract, Platine erklärt, Rechnungen überarbeitet und generelle Verbesserungen
52	10.03.26	3,0	Besprochene Änderungen in Dokumentation anpassen und Signalzeitdiagramm erstellen.	Signalzeitdiagramm erstellt
53	12.03.26	3,0	Dokumentation überarbeiten	zusätzlich noch Programm auf Blinker gespielt. Ein Blinker ist angepasst.
54	13.03.26	5,0	Dokumentation überarbeiten	Erfolgreich

Tabelle 18: Tagesziele Teil 3

7.5 Abbildungsverzeichnis

Abbildung 1: Kommunikation zwischen den Projekten im Sequenzdiagramm.....	6
Abbildung 2: Kommunikationsdiagramm des CAN-Protokolls.....	8
Abbildung 3: Cockpitplatine mit allen Geräten angeschlossen.....	9
Abbildung 4: Schaltplan der Cockpitplatine Rev 2.....	10
Abbildung 5: Bauteilansicht (top) der Cockpitplatine Rev 2.....	11
Abbildung 6: Leiterbahnen (bottom) der Cockpitplatine Rev 2.....	11
Abbildung 7: Bluetooth-Modul auf der Cockpitplatine.....	12
Abbildung 8: Alle vier Blinker-Module.....	13
Abbildung 9: männlicher CAN D-Sub Anschluss nach CiA® 106 (Quelle: PCAN-USB Anleitung)...	16
Abbildung 10: Stecker zum Programmieren des ATmega16M1.....	17
Abbildung 11: Oberseite des ATmega16M1.....	17
Abbildung 12: Unterseite des ATmega16M1.....	17
Abbildung 13: Anschlüsse des AT90CAN32 (Quelle: Schaltplanausschnitt von MS-001).....	18
Abbildung 14: Masterplatine mit AT90CAN32 (MS-001).....	18
Abbildung 15: Masterplatine mit AT90CAN32 (MS-001).....	18
Abbildung 16: ISP Pinbelegung (Quelle: wikipedia.org von osiixy erstellt).....	19
Abbildung 17: Masterplatine mit serieller und ISP Verbindung.....	20
Abbildung 18: Bluetooth-Modul mit HC-06 Oberseite (Quelle: roboter-bausatz.de).....	22
Abbildung 19: Bluetooth-Modul mit HC-06 Unterseite (Quelle: roboter-bausatz.de).....	22
Abbildung 20: Bluetooth-Modul Schaltung mit Arduino.....	24
Abbildung 21: Blinker auf Go-Kart positioniert.....	26
Abbildung 22: Innenleben des Blinker-Moduls hinten rechts.....	26
Abbildung 23: Signalzeitdiagramm für Blinker-LEDs.....	27
Abbildung 24: CAN bus elektrische Beispiel Topologie mit Abschlusswiderständen (Quelle: Stefan-Xp Wikipedia Nutzer).....	30
Abbildung 25: komplettes CAN frame Beispiel (Quelle: Canis Automotive Labs Ltd).....	30
Abbildung 26: Blinker-Modul mit D-Sub Kabel.....	31
Abbildung 27: männlicher CAN D-Sub Anschluss nach CiA® 106 (Quelle: PCAN-USB Anleitung). 31	
Abbildung 28: PAP zu dem CAN Testprogramm.....	32
Abbildung 29: Nextion Display (NX8048K070_011C).....	34
Abbildung 30: Ersatz Bauteil suche für Spannungsregler (Quelle: https://smd.yooneed.one).....	34
Abbildung 31: Cockpit-Display (NX4024K032).....	35
Abbildung 32: Miniatur Summer 24V (EMS-24L).....	39
Abbildung 33: Kennlinien (Quelle: Datenblatt TIP120).....	40
Abbildung 34: Lenkrad Taster.....	43
Abbildung 35: Start/Stop/Rückwärts.....	43
Abbildung 36: Taster-Anschluss.....	43
Abbildung 37: Entwurf der Cockpitplatine (Rev1).....	44
Abbildung 38: Schaltplan der Cockpitplatine Rev 2.....	45
Abbildung 39: Leiterbahnen (bottom) der Cockpitplatine Rev 2.....	46
Abbildung 40: Bauteilansicht (top) der Cockpitplatine Rev 2.....	46
Abbildung 41: Abmessung Ausdruck.....	47
Abbildung 42: Bestückte Platine.....	49
Abbildung 43: gefräste Kupferplatte.....	49
Abbildung 44: PCAN-USB (Quelle: peak-system.com).....	56

7.6 Schaltplanausschnitte

Schaltplanausschnitt 1: Anschluss des Bluetooth-Modul an die Masterplatine.....	24
Schaltplanausschnitt 2: Power-LED.....	42

7.7 Tabellenverzeichnis

Tabelle 1: Team.....	6
Tabelle 2: CAN-Protokoll.....	7
Tabelle 3: Cockpit Bluetooth-Modul Einstellungen.....	12
Tabelle 4: Blinker Bluetooth-Modul Einstellungen.....	13
Tabelle 5: CAN-Bus Einstellungen.....	16
Tabelle 6: Befehle für das Cockpit-Display.....	36
Tabelle 7: Prüfprotokoll von der Cockpit-Platine.....	48
Tabelle 8: Arbeitszeit Kosten.....	57
Tabelle 9: Material Kosten.....	57
Tabelle 10: alte Projekte.....	60
Tabelle 11: Terminplan 01.09. - 19.09.2025.....	61
Tabelle 12: Terminplan 20.10. - 07.12.2025.....	62
Tabelle 13: Terminplan 08.12.2025 - 25.01.2026.....	63
Tabelle 14: Terminplan 26.01. - 15.03.2026.....	64
Tabelle 15: Meilensteine.....	65
Tabelle 16: Tagesziele Teil 1.....	66
Tabelle 17: Tagesziele Teil 2.....	67
Tabelle 18: Tagesziele Teil 3.....	68

7.8 Quellenverzeichnis

AW01: Arch Wiki Bluetooth, 2026, <https://wiki.archlinux.org/title/Bluetooth>